7.UART

通用异步接收器/发送器，通常称为**UART**，是一种广泛应用于嵌入式领域的**串行、异步、全双工通信协议。**

物料：

usb转ttl，蓝牙模块

推荐：

主从一体蓝牙模块BT36(教程使用，第二个选项开发套件中包含usb转ttl)

实验室也有一定数量的物料提供借用，但数量有限，先到先得

一、通信方式简介

通信就是数据在双方或多方之间的准确传递，对于我们的二进制硬件系统来说，数据以二进制流（一串01101010…）的形式进行传递。

按数据传输方式分类：

1.串行通信：

数据在一条线上“排队”等待发送，类似于火车检录时只有一个检录口

2.并行通信（用不到，了解即可）

数据可以在多条线上同时进行发送，类似于火车的多个检录口，或是电路中的并联

按数据的传输方向分类：

这里我们所说的通信都是只有两方的

1.单工：

一方只接收，另一方只发送，比如电脑与打印机之间，打印机只接收来自电脑的数据，而不给电脑发数据

2.半双工：

两方均既可接收也可发送，但不能同时发送或接收

3.全双工

两方均既可接收也可发送，并且可以同时进行

按照数据的同步方式分类：

1.同步通信：

当使用同步通信时，双方要进行时间的对齐，这时候双方就可以进行时间上的约定“我下午三点要给你打电话“，如果没有对齐时间信号，就好像一个人在北京，而另一个人在纽约，并且他们不知道对方所处的时区，这时候上面的约定就会无效。

2.异步通信：

当使用异步通信时，双方不需要时间上对齐，但与此相对的，我们还是需要让对方知道我们什么时候发了消息，这时候发送消息的开始和结束都需要有一个约定好的特定信号，来标志我们将开始数据的传输，同时还必须约定好通信的速率，用来设置采样频率以及检测这段时间发了多少数据。

按照数据在线上的传输信号分类：

1.单端信号：

单端信号只使用一根信号线进行传输。由于**信号每一点的电压都是相对于地而言的**。在长距离传输时，电磁干扰对其影响较大，这就导致了信号的误差，即不稳定因素。加之可能会有干扰信号，这就导致了单端信号的抗干扰能力较差，同时需要公共的参考电平（一般为GND）

2.差分信号（双端信号）

差分信号使用两根信号线传输信号。在单端信号中，01流非常直观，与gpio相同，1代表高电平，0代表低电平，而使用差分信号时则是使用两根线上的电压差来表示，此时如果在某一点有干扰信号，这些误差会同时同方向地作用在两根信号线上，而电压差不会被改变，这样很好的抵抗了干扰。至于表示0/1的方式，具体通信协议不同，其规定也不同。同时，这种方式不需要公共的参考电平

上面我们对通信方式做了一个概述，我们最常用的串口（UART）按上述可以归类为串行，全双工，异步，单端。了解这些有利于对串口外设的寄存器设计有更加透彻的理解。

二、UART简介

UART（Universal Asynchronous Receiver/Transmitter），中文为通用异步收发传输器，

电平标准

UART有多种电平标准，我们最为常用的是ttl电平，高电平3V3对应二进制中的1，低电平0V对应二进制中的0。

接线

我们在上文中将其归类为串行，全双工，异步。从这之中我们可以推断出它如何接线：

异步，所以通信双方不需要同步时钟信号，因此不需要时钟线

全双工，收发可以同时进行，那么自然不能在同一根线上，所以数据传输至少需要两根线（一收一发）

串行，单端：数据在一个窗口上排队等候发送，并且只需要一根线，所以无论收发都只有一根线（收为RX,发为TX），此外单端还需要参考电平，所以需要将GND相接

综上，串口只需要RX,TX,GND三条线即可完成数据传输，一般地，当两方均有独立供电时不接3V3（如果接了可能会导致板子内部集成的降压模块被烧毁），当其中一方没有独立供电时，将3V3（电源）相接。

VCC----VCC(可省略除非需要供电)

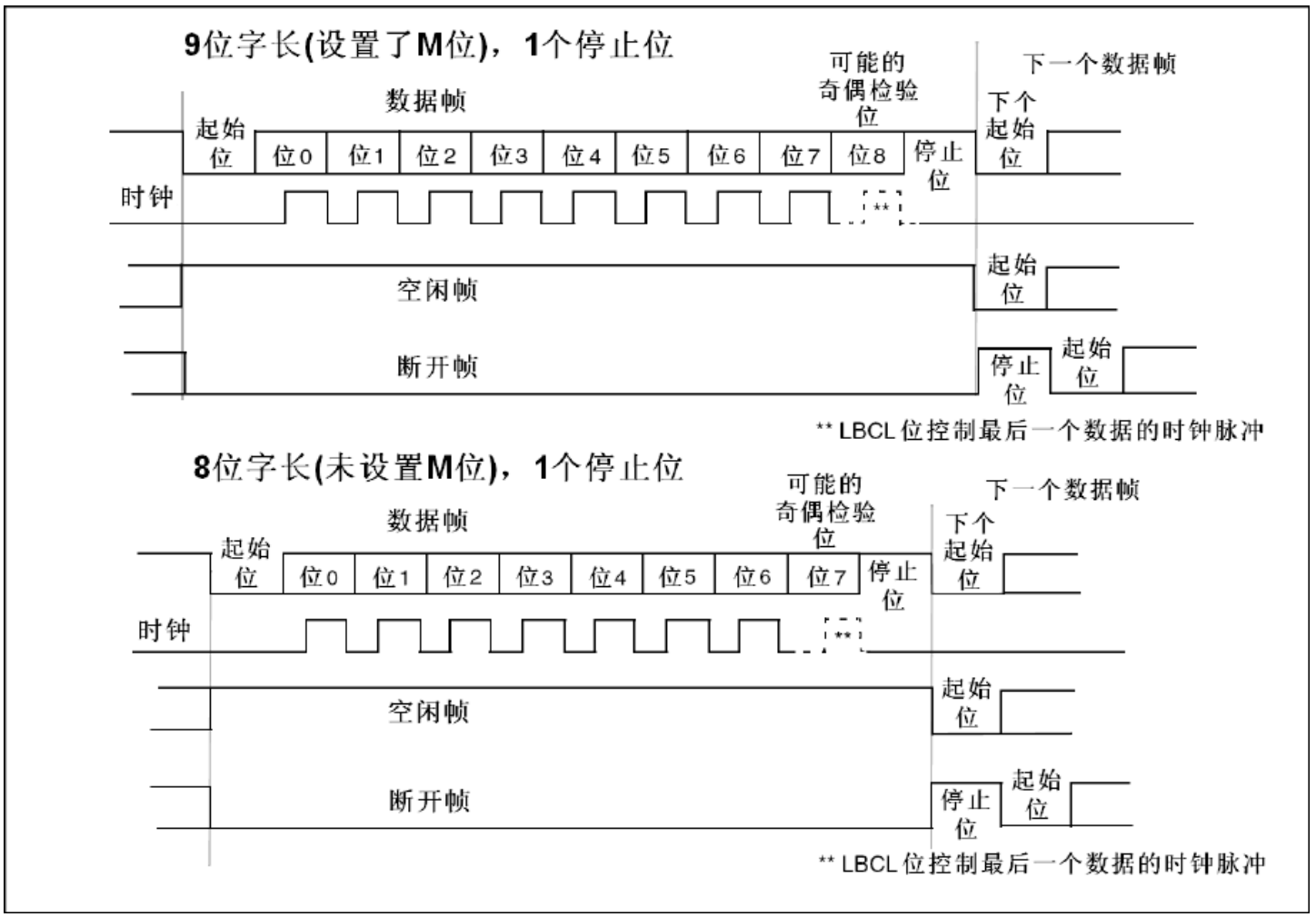
GND---GND

RX----TX

TX----RX

通信参数和流程

UART采用异步通信方式，有着起始位，停止位，并且需要规定其波特率，为了确保数据在传输过程中没有受到干扰，一般还有一位校验位。其中起始位固定为1位，数据位可选8或9位，若有校验位，则会包含在数据位中，停止位为1/1.5/2位（可选），停止位长度没有特定含义，双方约定一致即可，这与我们上文说到的异步通信的特点相吻合，其需要特定信号来标志信息的开始和结束，下图为uart的数据包结构：



当线上没有数据时，它会维持高电平。当需要发送数据时，发送端拉低电平一个周期作为起始位,接收端会利用这个起始位来校准时钟,此后,接收端根据每一段中点的位置进行采样,发送端一次发送数据帧和校验位,最后拉高1或2个周期作为停止位,然后发送下一个起始位,同样,接收端校验并正确的收到停止位后,会准备接收下一个数据帧

(注意,上面的图有一定的错误,实际上时钟是过采样的,也就是时钟的频率会是波特率的16或者8倍,最终采样的会是中间那一个时刻)

UART的校验分为无校验，奇校验，偶校验三种，下图为校验方式说明：

名称	数据	校验位	1的奇偶	说明
奇校验	11001110	0	奇数个1(5)	数据及校验位共计奇数个1
	11001100	1	奇数个1(4)	
偶检验	11001110	1	偶数个1(5)	数据及校验位共计偶数个1
	11001100	0	偶数个1(4)	
0校验	xxxxxxxx	0	/	Space,校验位总为0
1校验	xxxxxxxx	1	/	Mark,校验位总为1
无校验	xxxxxxxx	/	/	noparity

数据的传输速率通常用波特率描述，单位是bit/s，即单位时间内传输的bit数（这里不严谨，但无需在意），一般地，波特率和比特率虽然是不同的概念，但其数值上是一样的，有兴趣可以看下面这段：

代码块

```
1. 比特率：Bitrate，每秒传输的二进制位数，单位为比特每秒（bit/s）。

2. 波特率：baudrate，每秒传输的码元数。

码元是通讯信号调制的概念，通讯中常用时间间隔相同的符号来表示一个二进制数字，这样的信号称为码元。如常见的通讯传输中，用0V 表示数字0，5V 表示数字1，那么一个码元可以表示两种状态0 和1，所以一个码元等于一个二进制比特位，此时波特率的大小与比特率一致；如果在通讯传输中，有0V、2V、4V 以及6V 分别表示二进制数00、01、10、11，那么每个码元可以表示四种状态，即两个二进制比特位，所以码元数是二进制比特位数的一半，这个时候的波特率为比特率的一半。因为很多常见的通讯中一个码元都是表示两种状态，人们常常直接以波特率来表示比特率，虽然严格来说没什么错误，但希望您能了解它们的区别。
```

重要寄存器

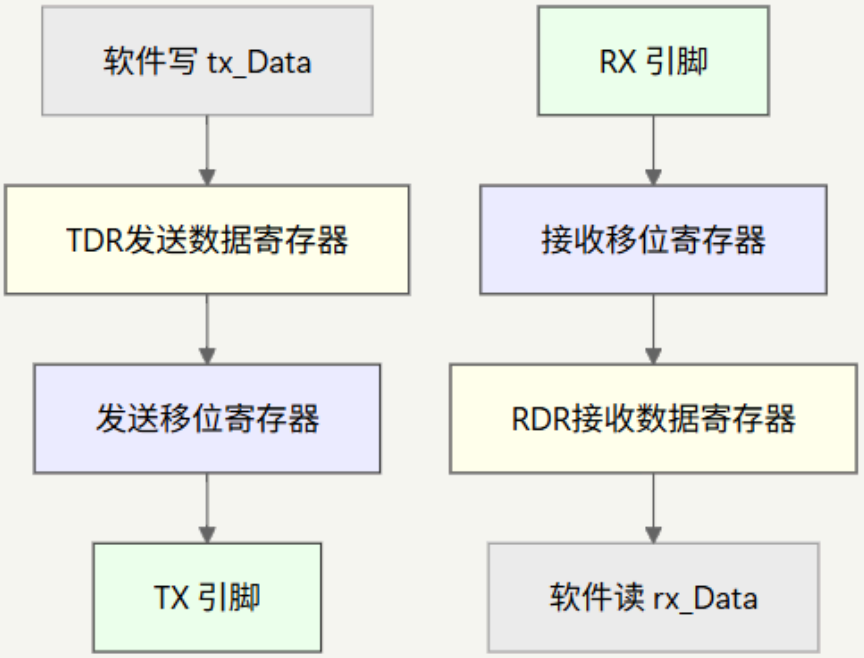
使用单片机其实就是在配置寄存器

在进行外设的使用时，我们最为关心的是外设的状态，所以对于串口，我们最关注的寄存器是它的状态寄存器USART_SR，这部分可以在STM32F1手册中的25.6节找到，这里给出该寄存器的精简解释

位	名称	含义	置1条件
0	PE	Parity Error	收到帧校验错
1	FE	Framing Error	停止位检测失败
2	NE	Noise Error	采样发现噪声
3	ORE	Overrun Error	接收移位寄存器已满且 RXNE 未读
4	IDLE	Idle Line Detected	检测到总线空闲（接收变高≥1 帧）
5	RXNE	Read Data Reg Not Empty	接收数据已搬到 RDR
6	TC	Transmission Complete	发送移位寄存器空（含停止位）
7	TXE	Transmit Data Reg Empty	TDR 已空，可写新数据
8	LBD	LIN Break Detected	检测到 LIN 断开
9	CTS	CTS Flag	nCTS 脚电平变化（硬件流控）

其中8/9位用不到，不做解释

下图为简化版的串口数据传输框图，在其中可以看出发送移位寄存器与接收移位寄存器在数据传输中的媒介作用（tx_data为发送数据的存储位置，rx_data为接收数据的存储位置）



准备发送的数据会先被送进发送缓冲寄存器（由CPU或者后面会学的DMA），然后压进移位寄存器一点点发出去，接收到的数据先在接收移位寄存器里被组装好，然后被搬进接收缓冲寄存器，由CPU或者DMA搬到指定的位置

PE,FE,NE都是字面意思，在数据传输过程中发生，不多做解释了，出现了查走线或者查配置

ORE，过写入，接收缓冲区里的上一条数据没有被收走，下一条又被写进来了，在接收中断里干重活会发生，所以接收和处理应该被分到不同的位置

IDLE，空闲，接下来连续一整节的时间里都是保持高电平，此时认为接收已经结束了，也就是IDLE，可以据此截出一段数据来，可以用它来实现不定长接收

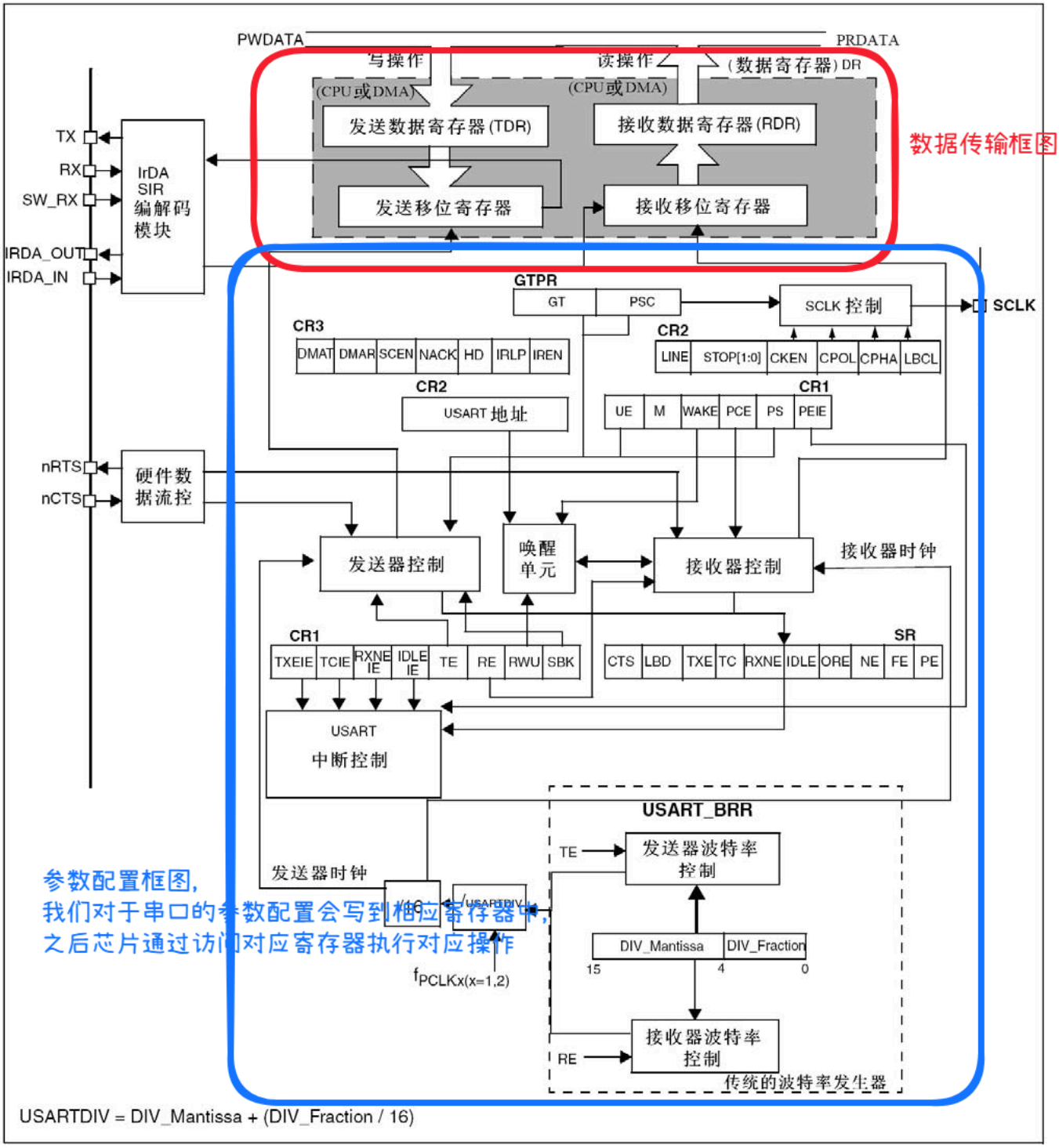
RXNE，TC，TXE也是字面意思

下为USART框图：

数据框图与上文简化版的基本一致

参数配置框图较为复杂，仅作了解即可，下文实战在配置cubemx时也将对其的一部分进行讲解

图248　USART框图



三、实战练习

这部分我们需要用到串口助手工具，串口助手工具可以使我们的电脑和单片机建立起通讯，我选了一个用着还不错的放在这里：

JComV1.5.0.zip
1.34MB

或者可以去JCOM官网下

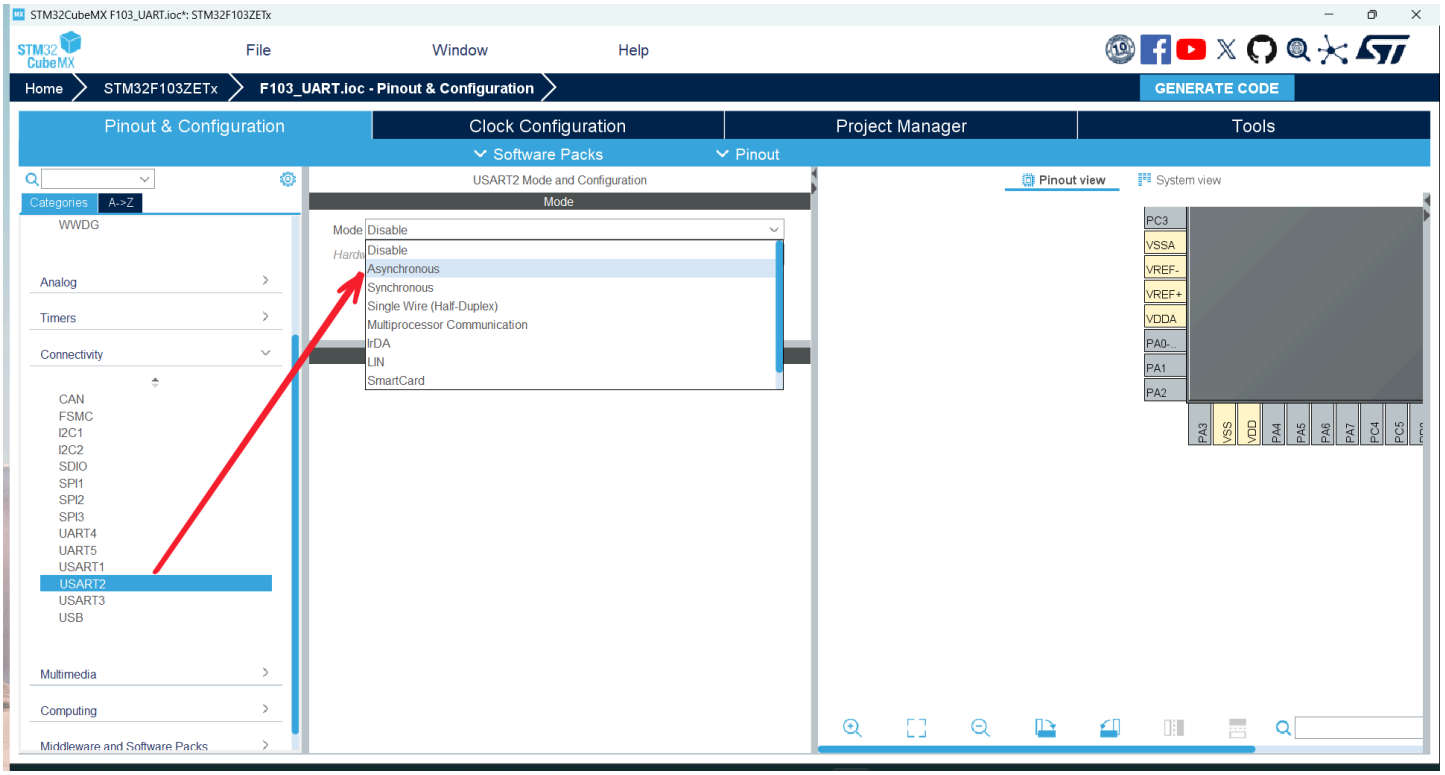
首先进入cubemx，选择自己的板子，我这里以f103zet6为例

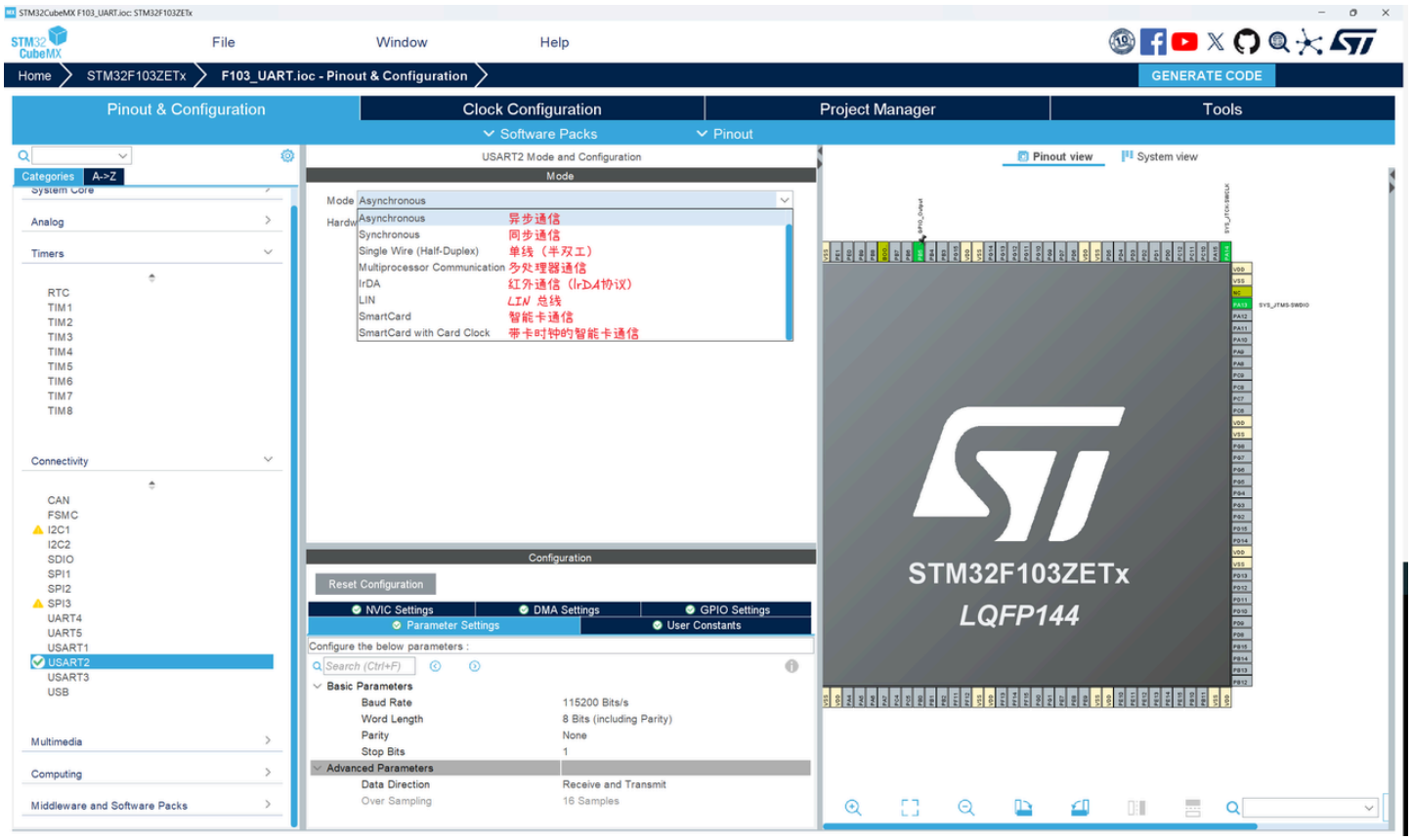
首先配置时钟与debug方式，前面已经进行过这里不加赘述

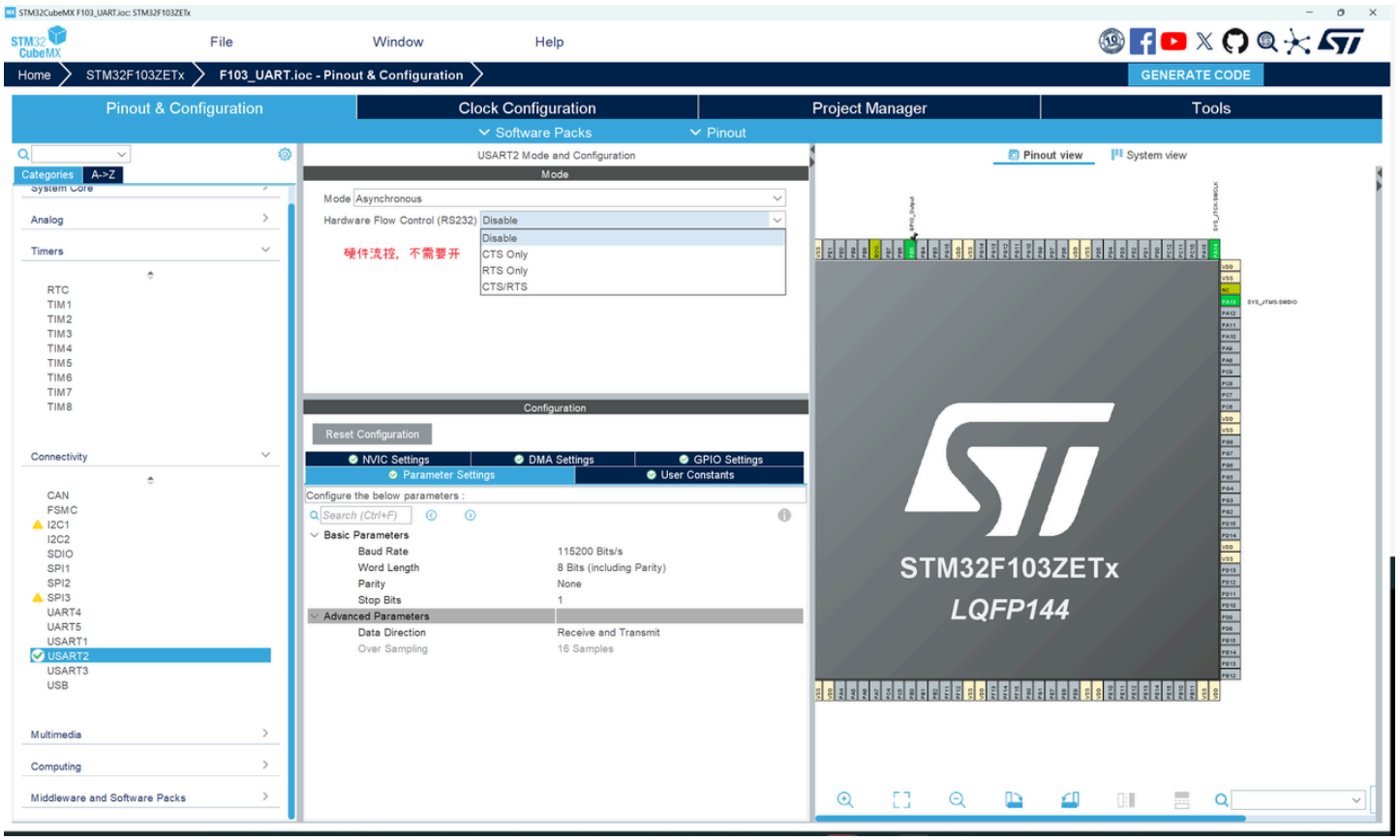
左侧栏选择connectivity,随意选择一个串口（UART与USART均可）

代码块

```
1    TIPS:USART与UART相比多了同步功能，但我们并不使用，依旧配置成异步模式即可，所以就我们的使用而言两者没有区别
```



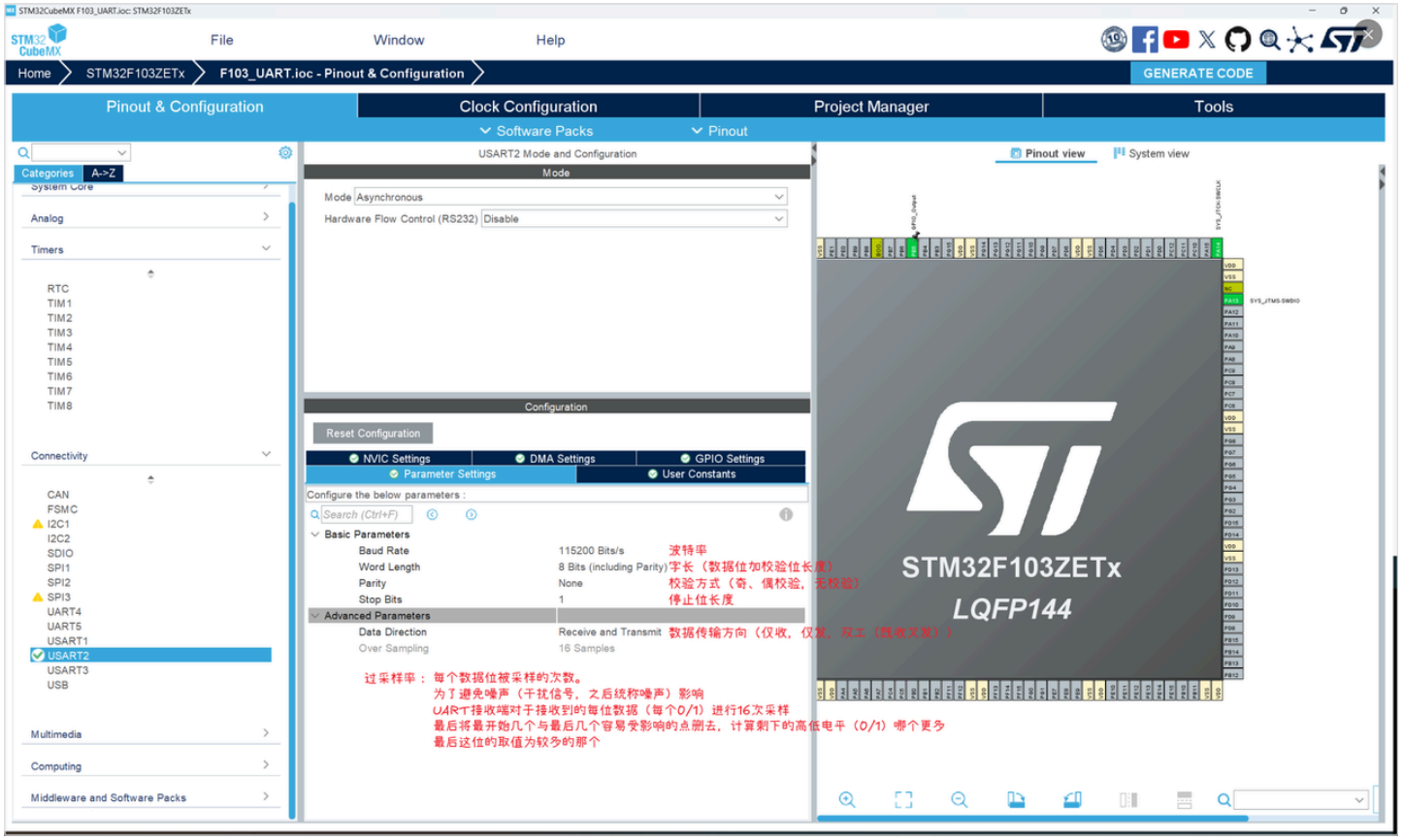




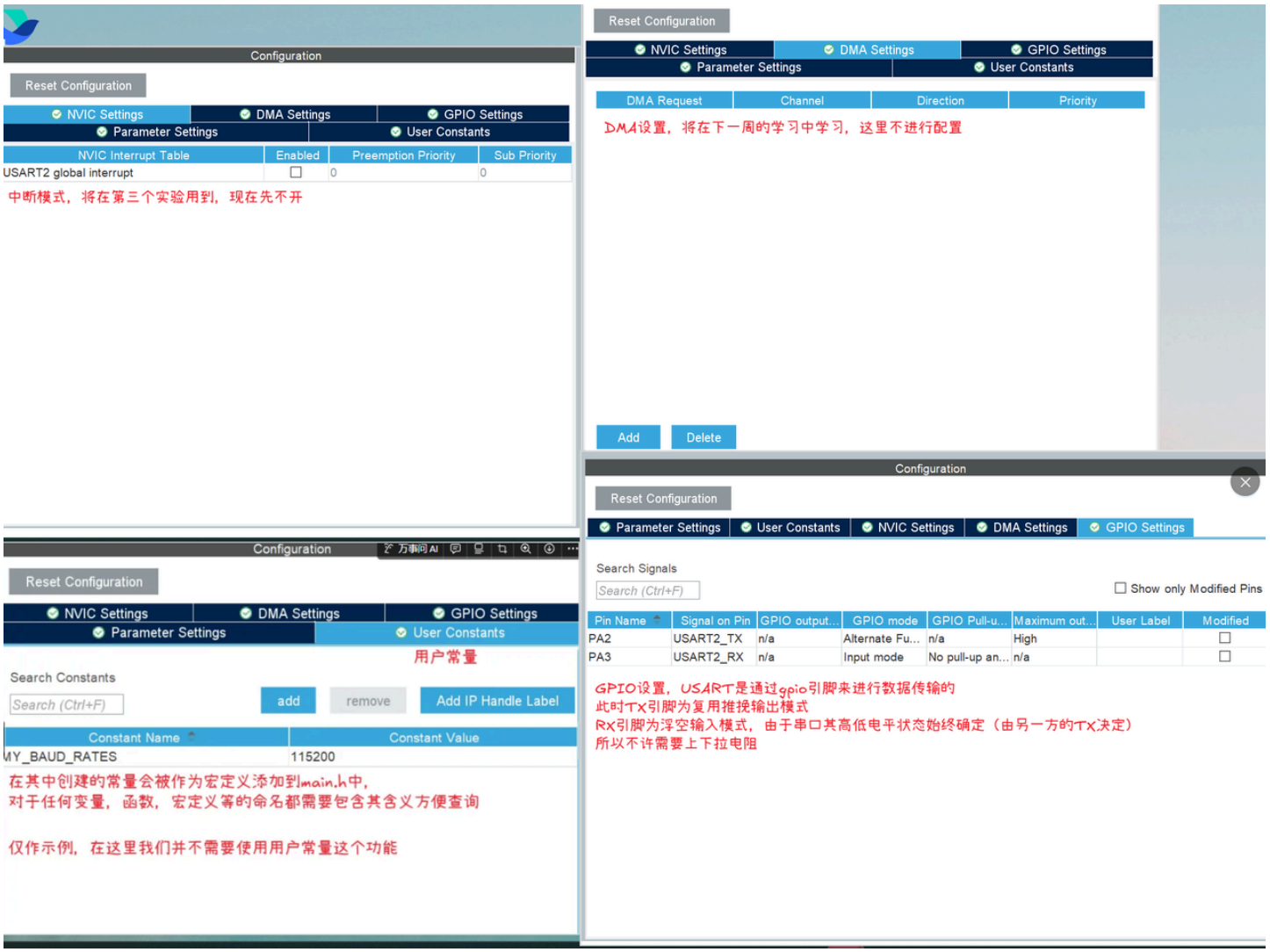
展开可以看到许多不同模式，这里给出翻译，但我们一般只用异步(**Asynchronous**)模式，这些特殊模式大多通过配置USART的CR3寄存器进行使能，如下：

位	名称	作用一句话
10	CTSIE	CTS 中断：1=开
9	CTSE	CTS 硬件流控：1=使能
8	RTSE	RTS 硬件流控：1=使能
7	DMAT	DMA 发送：1=开
6	DMAR	DMA 接收：1=开
5	SCEN	智能卡模式：1=开
4	NACK	校验错发 NACK：1=使能
3	HDSEL	半双工：1=单线模式
2	IRLP	IrDA 低功耗：1=低功耗
1	IREN	IrDA 模式：1=开
0	EIE	帧/噪声/溢出错误中断：1=开

接下来点击parameter setting进行参数配置，其中除过采样率外我们在上文中均已介绍到，其中波特率，字长，校验方式，停止位长度在硬件支持的基础上与通信另一方约定好即可（波特率不宜过高过低），数据传输过慢，可能导致在规定时间传不完，过快可能导致数据出错，一般选用115200。

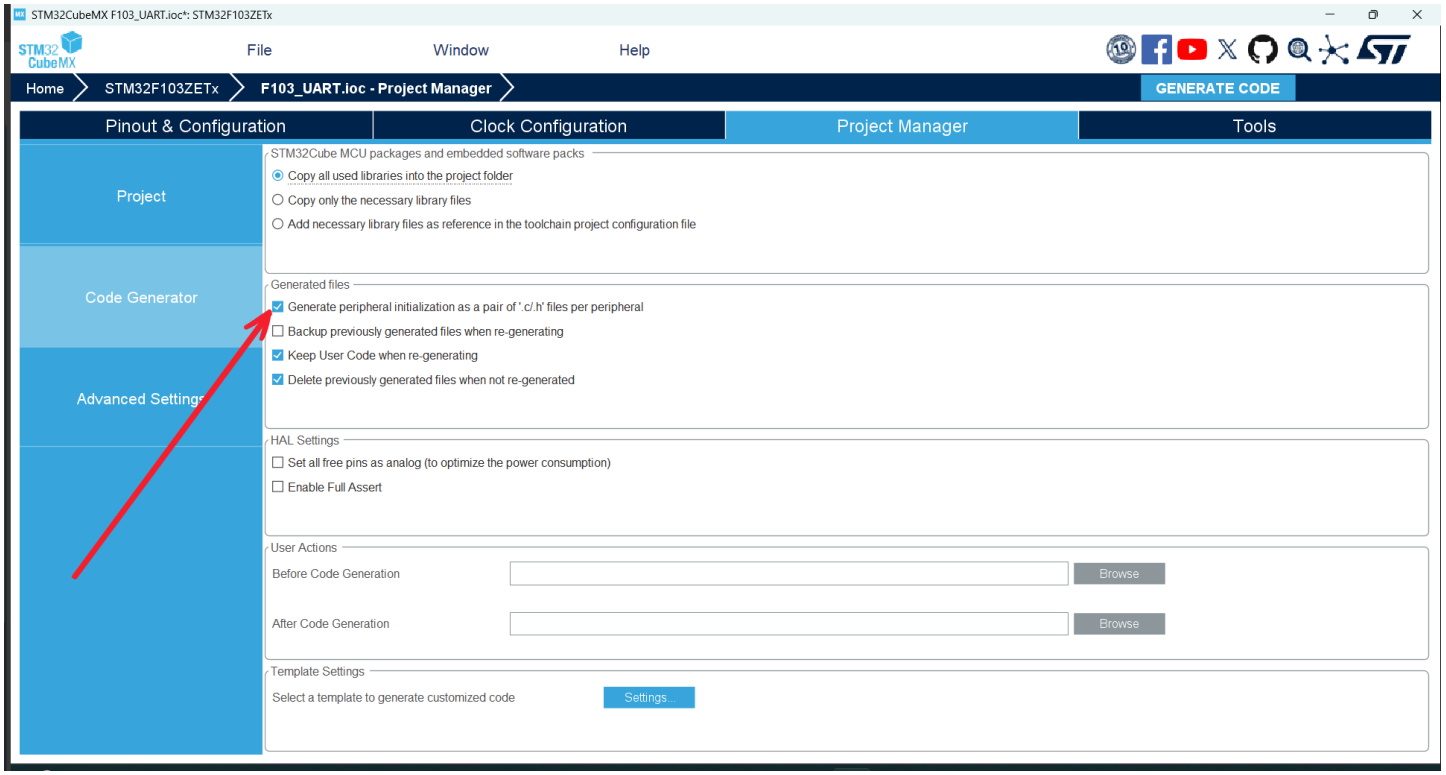


对于另外几个选项卡，这里放一张组合图简单介绍：



(关于用户常量,如果不需要可以不写,不影响系统本身的配置)

随后生成工程，生成之前勾选Generate peripheral initialization as a pair of'.c/.h files per peripheral（为每个外设生成一对‘.c/.h’文件初始化外设），这可以让我们的代码结构更加清晰



打开工程，在main函数中我们可以看到cubemx预先生成的 MX_USARTx_UART_Init(),函数内容如下：

代码块

```
1  void MX_USART2_UART_Init(void)
2  {
3
4    /* USER CODE BEGIN USART2_Init 0 */
5
6    /* USER CODE END USART2_Init 0 */
7
8    /* USER CODE BEGIN USART2_Init 1 */
9
10   /* USER CODE END USART2_Init 1 */
11   huart2.Instance = USART2;//使用哪个串口
12   huart2.Init.BaudRate = 115200;//波特率设置
13   huart2.Init.WordLength = UART_WORDLENGTH_8B;//数据位长度设置
14   huart2.Init.StopBits = UART_STOPBITS_1;//停止位长度设置
15   huart2.Init.Parity = UART_PARITY_NONE;//校验设置
16   huart2.Init.Mode = UART_MODE_TX_RX;//数据传输设置
17   huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;//硬件流控设置（不需要知道是什么，现在
     也没用）
18   huart2.Init.OverSampling = UART_OVERSAMPLING_16;//过采样率设置
```

```
19    if (HAL_UART_Init(&huart2) != HAL_OK)//初始化
20    {
21      Error_Handler();
22    }
23    /* USER CODE BEGIN USART2_Init 2 */
24
25    /* USER CODE END USART2_Init 2 */
26
27  }
```

可以看到我们在cubemx中配置的参数在这里被初始化了，之后又进入了HAL_UART_Init()函数，在这之中将这些参数写入到对应寄存器中，由于内容太多，我们仅举例说明：

代码块

```
1  UART_SetConfig(huart);//进去后又执行这个函数
2
3  MODIFY_REG(huart->Instance->CR2, USART_CR2_STOP, huart->Init.StopBits);
4  //可以看到停止位长度被写到了CR2寄存器中的某位，下文给出CR2寄存器：
```

位	名称	作用一句话
14	LINEN	LIN 模式：1=使能
13:12	STOP	停止位：00=1；01=0.5；10=2；11=1.5
11	CLKEN	同步时钟输出：1=使能 CK 引脚
10	CPOL	时钟极性：0=低空闲，1=高空闲
9	CPHA	时钟相位：0=第 1 沿采样，1=第 2 沿采样
8	LBCL	最后一位时钟脉冲：1=输出
6	LBDIE	LIN 断开检测中断：1=开

5	LBDL	断开长度：0=10 位，1=11 位
3:0	ADD\[3:0]	多处理器地址/智能卡地址

这里将停止位长度写入了USART_CR2中的STOP，具体写入过程暂不深究

通过以上，我们完成了使用UART的前置准备过程，并且看到了我们的配置与寄存器之间 的联系

轮询发送与接收

轮询模式即不断访问相应寄存器，当符合条件时执行，在执行前只要没达到超时时间就会一直等待并持续访问相应寄存器

这里我们主要使用两个函数：

代码块

```
//发送函数
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, const uint8_t *pData, uint16_t Size, uint32_t Timeout)

//接收函数
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
```

对于HAL_UART_Transmit，其在超时时间内反复访问USART_SR中的TXE，直到发送出指定长度的数据

对于HAL_UART_Receive，其在超时时间内反复访问USART_SR中的RXNE，直到接收到指定长度的数据

可见,我们只要事先配置好串口,然后直接调用这俩函数,就可以直接启动定长收发了

接下来我们对这两个函数进行解读

参数含义如下：

参数名	类型	含义说明
`huart`	`UART_HandleTypeDef *`	指向 UART 的指针，包含了 UART 的信息（如波特率、数据位、停止位等）。例如：`&huart1` 表示使用 UART1。
`pData`	`const uint8_t*`	指向数据的指针（transmit中为要发送的数据的指针，receive中为数据要接收到的地方）。
`Size`	`uint16_t`	数据字节数（ransmit中为要发送的数据的长度，receive中为要接收的数据长度）。
`Timeout`	`uint32_t`	超时时间，单位为**毫秒（ms）**。如果在这段时间内数据没有发送完成，函数会返回 `HAL_TIMEOUT`。

返回值为HAL_StatusTypeDef类型的，其定义如下，暂时不用理解其含义，只需要知道其会返回HAL_OK/…四个中的一个即可

代码块

```
typedef enum
{
  HAL_OK       = 0x00U,
  HAL_ERROR    = 0x01U,
  HAL_BUSY     = 0x02U,
  HAL_TIMEOUT  = 0x03U
} HAL_StatusTypeDef;
```

首先对要使用的变量进行声明，将变量声明在函数之外时，这个变量是全局变量，全局变量便于我们debug时查看，char*与uint8_t*可以通过强制转化实现互用，所以tx_data即使不是uint8_t*类型的，也可以作为传参

Transmit:

这里超时时间为500ms，如果超时则返回HAL_TIMEOUT

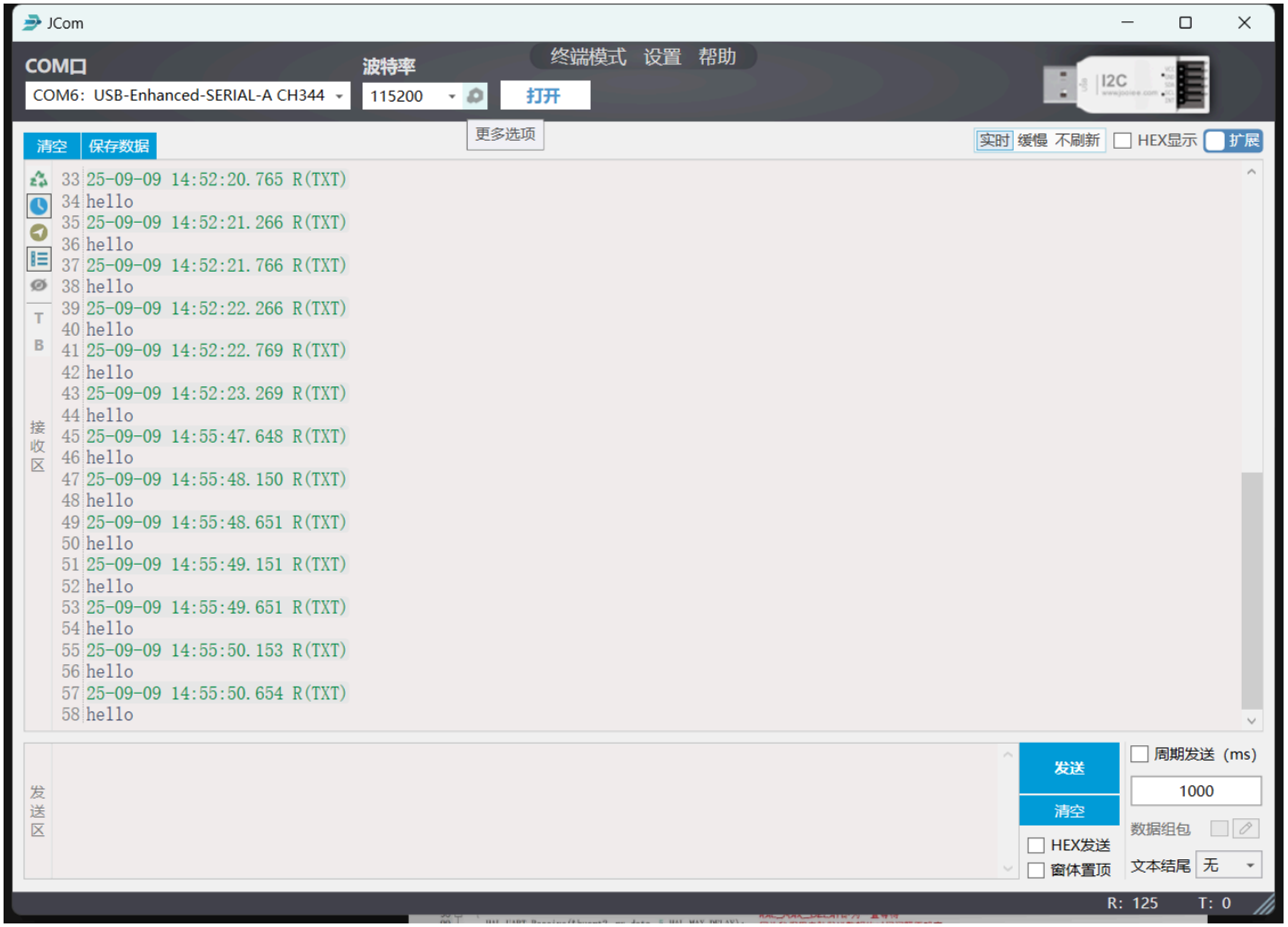
```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    HAL_UART_Transmit(&huart1,(uint8_t*)tx_data, Size: sizeof(tx_data), Timeout: 500);
    //这里我用的huart1,你根据你实际开的串口来写)
    //这个函数意思是阻塞发送,不断尝试发送直到发完或者超时,还有后缀it的,意思是中断发送,后缀dma,意思是由dma搬运
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    HAL_Delay(500);
  }
  /* USER CODE END 3 */
}
```

(上面那个参数表别写Size:和Timeout:哈,那个是自动补全的结果,下同)

烧录到单片机，用usb转ttl连接电脑和单片机，打开串口助手，选择你的com口（大概率是ch340那个，与使用的usb转ttl型号有关）和波特率，点击更多选项，如果数据包格式不一样则修改一致，单击打开，看到消息证明实验成功。





这里有的人可能会收到残缺的消息”hel”这是因为你使用的声明方式是char* tx_data = “hello”;此时调用sizeof(tx_data)，会得到指针所占内存的大小4（字节），-1后则是3，这就是二者的不同之一，这里我们可以选择使用strlen(tx_data)，它直接获取的是字符串中字符的数量，需要注意，使用strlen时必须先#include “string.h”，

放到如下代码区：

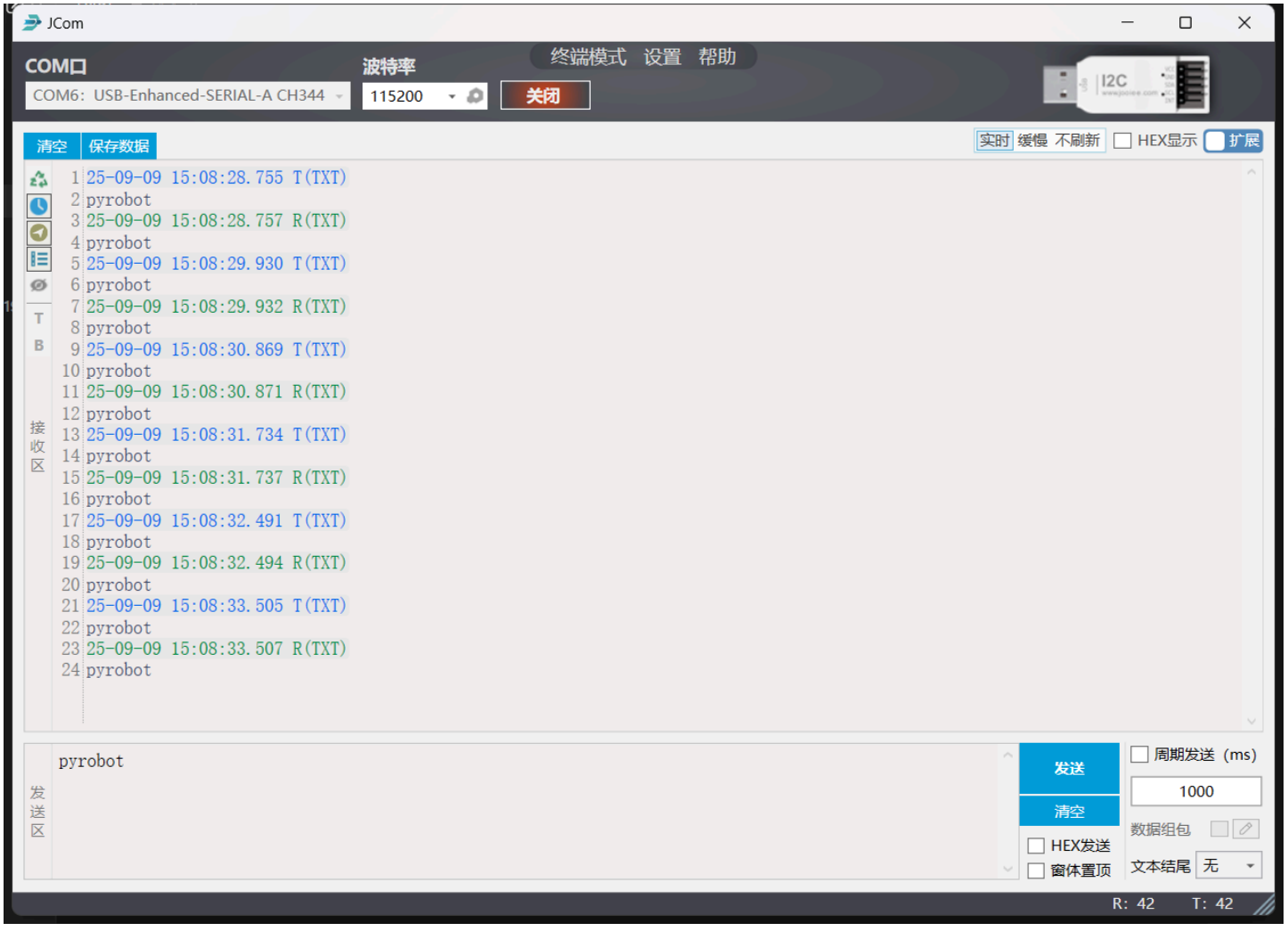
代码块

```
/* Private includes ----------------------------------------------------------
*/
/* USER CODE BEGIN Includes */
#include "string.h"
/* USER CODE END Includes */
```

这里进一步解释一下,对于一个数组,取它的尺寸会得到它真正的尺寸,但是上面提到的是指针,32位机意味着寻址范围是32位,也就是一个指针大小是32位,也就是4字节,直接取指针的尺寸就取出4来了

Receive:

```
95    char tx_data[7]="";
96    char rx_data[7]="";
97    /* USER CODE END 2 */
98
99    /* Infinite loop */
100   /* USER CODE BEGIN WHILE */
101   while (1)
102   {
103     memset(tx_data,'\0',7);
104     HAL_UART_Receive(&huart1,(uint8_t*)rx_data, Size: sizeof(tx_data), Timeout: HAL
105     memcpy(rx_data,tx_data,7);
106     HAL_UART_Transmit(&huart1,(uint8_t*)tx_data, Size: sizeof(tx_data), Timeout: 50
107     /* USER CODE END WHILE */
108
109     /* USER CODE BEGIN 3 */
```



也可以尝试一个个字符敲进去,刚才接收是最大阻塞,也就是一直填满7个字符才会结束接收

接收不定长数据

发送不定长数据很简单，只需要每次发送前计算要发的数据长度就可以，但接收的数据并不在单片机上，在收到前难以知道其长度，因此要接收不定长数据，我们必须采用其他方式

前面我们提到使用单片机就是在配置寄存器，HAL_UART_Receive实际检测的是RXNE位，回到上面对寄存器的讲解，我们可以发现有一位可以帮助我们接收不定长数据，即IDLE位，我们可以利用这位进行判断，当IDLE位被置1（空闲）时，我们认为对方完成了一次数据的传输，这时我们并不需要预先知道字节数，对于这个，HAL库也提供了封装好的函数

代码块

```
1   HAL_StatusTypeDef HAL_UARTEx_ReceiveToIdle(UART_HandleTypeDef *huart, uint8_t
    *pData, uint16_t Size, uint16_t *RxLen,  uint32_t Timeout)
```

此时虽然也需要传入size，但这个代表最大字节数，当达到这个最大字节数时，无论总线是否空闲，都会强制停止，一般用传入数组的最大长度

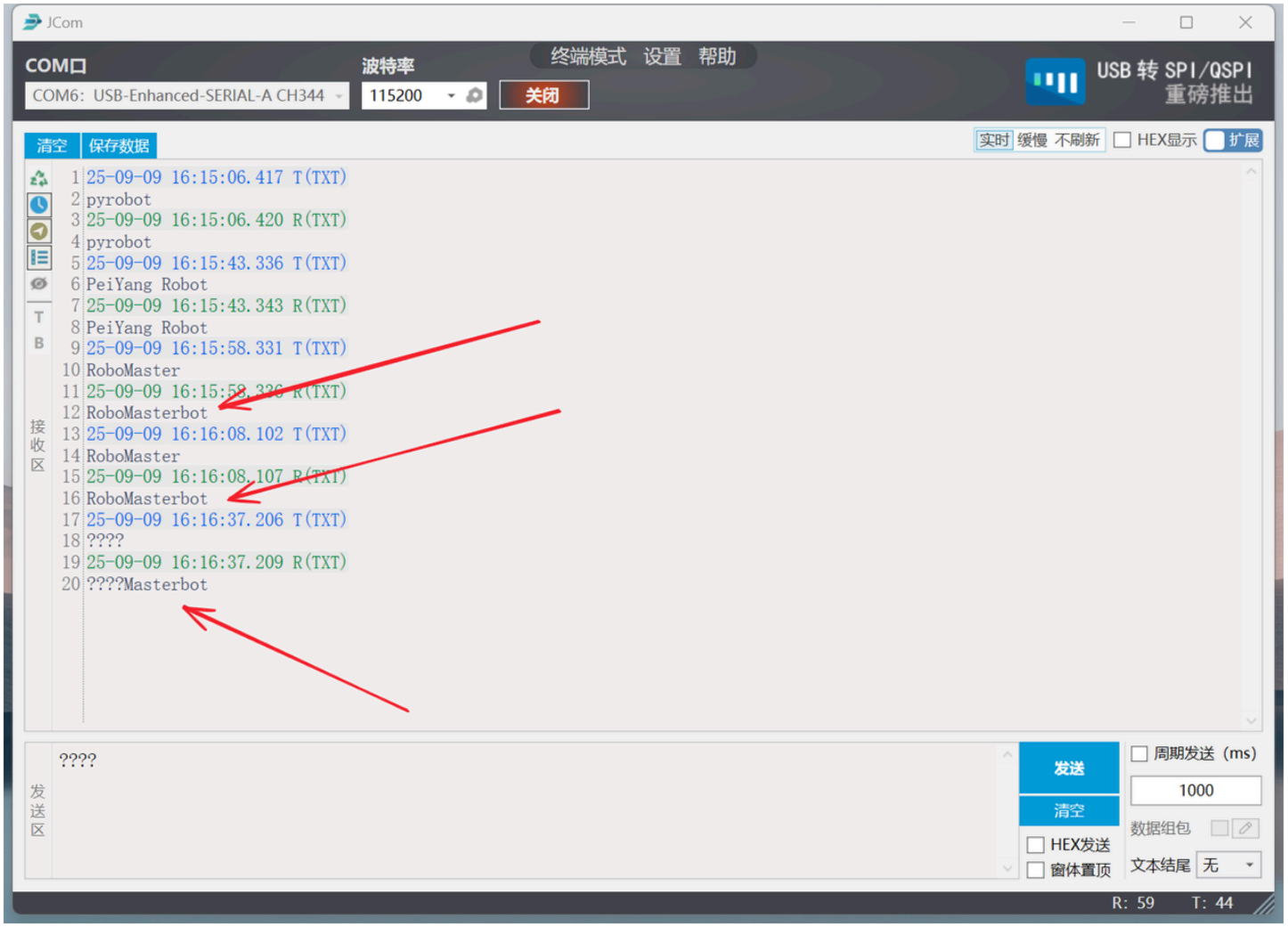
参数含义如下：

参数名	类型	含义
`huart`	`UART_HandleTypeDef *`	指向已初始化的 UART 句柄，包含寄存器基地址、配置、状态等。
`pData`	`uint8_t *`	用户提供的空间，用于存放接收到的字节。
`Size`	`uint16_t`	最大容量（单位：字节），也是一次调用最多允许接收的字节数。
`RxLen`	`uint16_t *`	函数返回时，RxLen 被写入“实际已接收到的字节数”。
`Timeout`	`uint32_t`	最长等待时间（毫秒）。在此时间内若未收满 Size 且未出现 IDLE 事件，则返回 `HAL_TIMEOUT`。

代码块

```
//增加变量
uint16_t rx_len;
//数组增大
uint8_t rx_data[100];
//将上个实验中的HAL_UART_Receive(&huart2,rx_data,7,HAL_MAX_DELAY)替换
HAL_UARTEx_ReceiveToIdle(&huart2,rx_data,100, &rx_len , HAL_MAX_DELAY);
```

烧录代码进行实验：



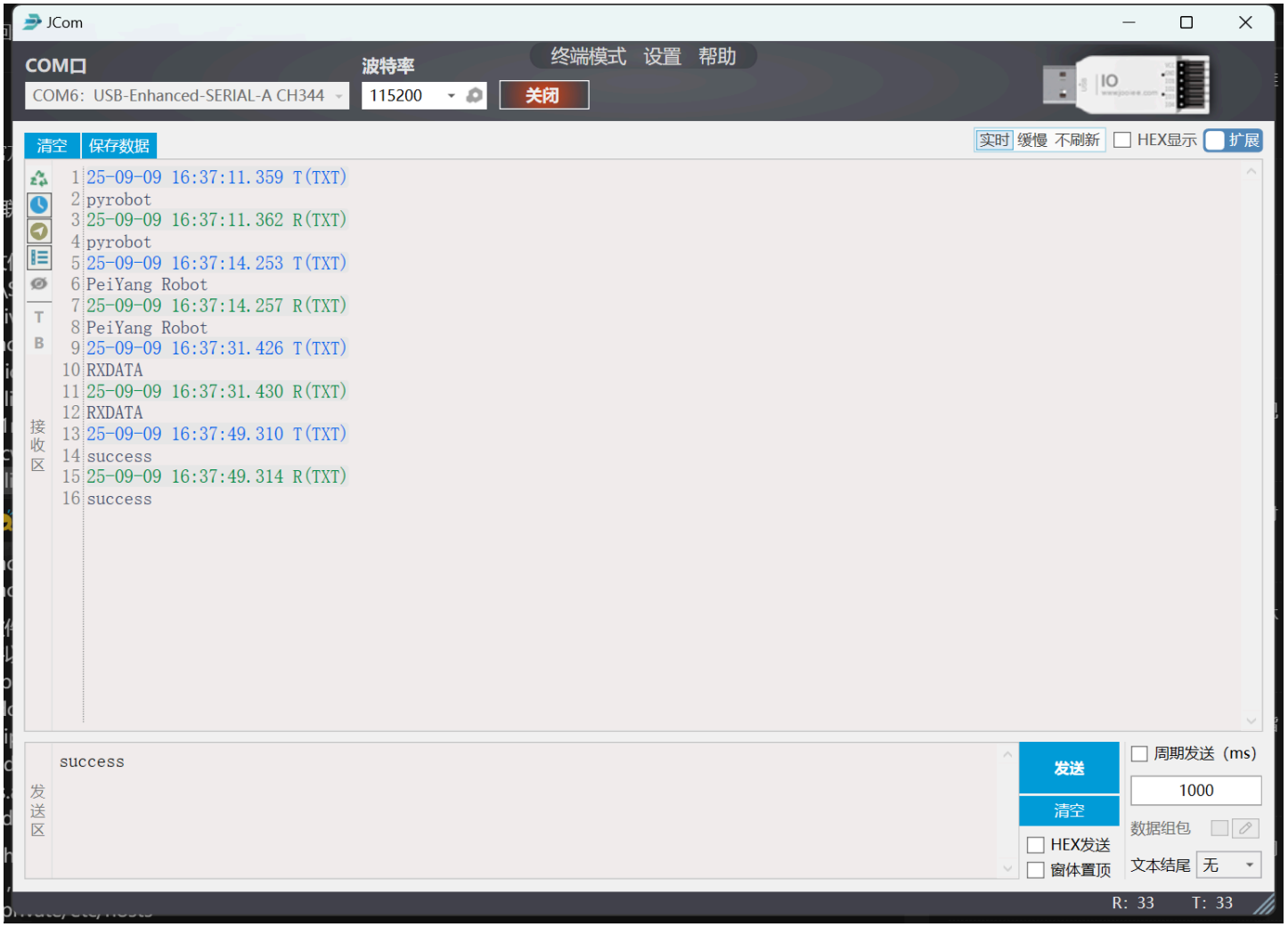
刚开始时一切正常，但后面我们却发现当输入长数据后，发回来的始终带着后面的内容，这是因为接收数据时只会把需要用到的地方覆盖，而不会对整个数组进行重写，这时候就体现出参数中rxlen的重要性了，借助这个，我们可以把不需要的数据擦除，也可以选择不发出

代码块

```
//擦除数据
for(int i = rx_len;(rx_data[i] != 0);i++)
{
    rx_data[i] = 0;
}

//不发出
HAL_UART_Transmit(&huart2,rx_data,rx_len,100);
```

再次烧录代码进行实验：

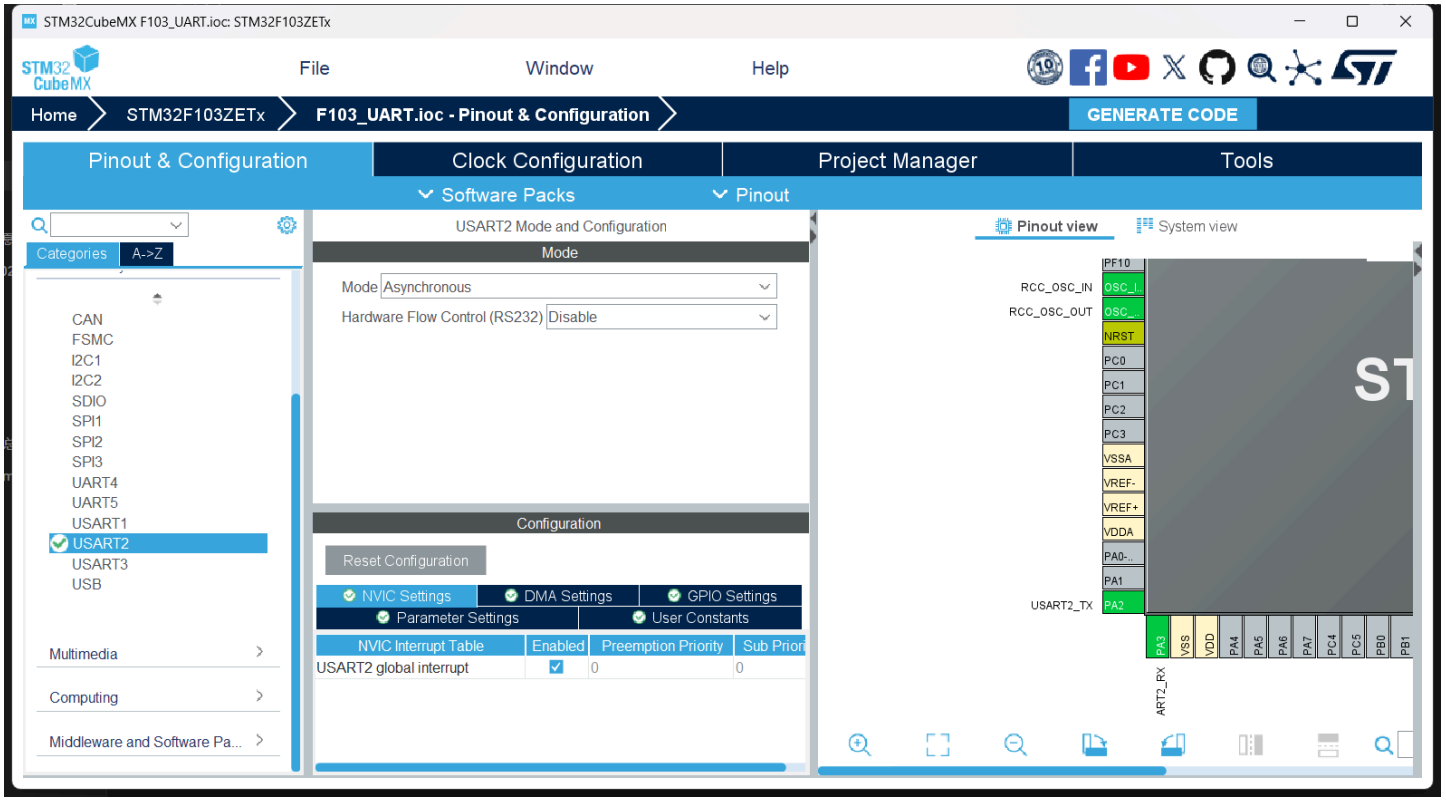


实验成功

中断收发

上文中我们使用的均为轮询模式进行收发，它的缺点是在执行时反复访问对应寄存器，这会将程序堵塞，当我想要在执行其他功能（如小灯翻转）的同时接收消息，这将会影响原本程序的运行，采用中断模式则可以解决这个问题。

先打开cubemx，打开中断，重新生成代码



经过了上一节NVIC的学习，想必大家已经认识了中断，回调函数等概念，下面不再进行重复说明

中断处理函数：（HAL_UART_IRQHandler）

如果没有开其他外设的话应该在_it文件的最下面可以找到图示位置

```
204  void USART1_IRQHandler(void)
205  {
206    /* USER CODE BEGIN USART1_IRQn 0 */
207
208    /* USER CODE END USART1_IRQn 0 */
209    HAL_UART_IRQHandler(&huart1);
210    /* USER CODE BEGIN USART1_IRQn 1 */
211
212    /* USER CODE END USART1_IRQn 1 */
213  }
214
215  /* USER CODE BEGIN 1 */
216
217  /* USER CODE END 1 */
218
```

所有串口的任何中断都会执行这个中断处理函数，同时进入这个函数的还有其句柄（就是uart的结构体指针），如上图就传入了&huart1，在其中会根据寄存器的标志位去判断是触发了哪个中断，从而执行相应的回调函数，这里我们最常用的回调有发送完成回调，接收完成回调以及事件回调，这里的事件回调只有在调用HAL_UARTEx_ReceiveToIdle_IT/DMA(DMA将在下节学到)后才会被使能，并且每次进入后都会失能，接收完成回调HAL_UART_Receive_IT/DMA后才会被使能，并且每次进入后都会失能。

跳转到HAL_UART_IRQHandler函数，我们可以看到其中有很多判断语句，他们将会判断总线是否发生错误，并选择要进入的回调函数

可以看到，在我们目前的状态下，最后会进入如下几个函数：

代码块

```
1   UART_Receive_IT(huart);
```

```
  HAL_UARTEx_RxEventCallback(huart, nb_rx_data);
  UART_Transmit_IT(huart);
  UART_EndTransmit_IT(huart);
```

看两个接收有关的函数

代码块

```
/*
 * __weak修饰的函数为弱定义函数，我们可以对其进行重写，重写后原本的定义失效
 * 这种弱定义方便用户基于库的二次开发
 * 这里我们可以看到该回调里没什么东西
 * UNUSED只是一种防止报错的方法，没有实际作用
 * 之后我们将对其进行重写，发生该中断时将会执行我们自己的代码
 */

__weak void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t
Size)
{
  /* Prevent unused argument(s) compilation warning */
  UNUSED(huart);
  UNUSED(Size);

  /* NOTE : This function should not be modified, when the callback is needed,
            the HAL_UARTEx_RxEventCallback can be implemented in the user file.
   */
}

/*
 * 对于UART_Receive_IT(huart)，其代码量同样较多，这里暂不进行逐行讲解
 * 跳转到函数中，我们可以看到众多判断语句，但这并不重要
 * 我将关键部分摘录如下（看不懂#if#else看下边）
 * 这部分与上文HAL_UART_IRQHandler的一部分十分相近
 * 归根结底，它判断了是空闲中断还是接收完成中断
 * 调用HAL_UART_Receive_IT接收完成时进入接收完成中断
 * 调用HAL_UARTEx_ReceiveToIdle_IT总线空闲时进入空闲中断
 * 之后分别进入到HAL_UARTEx_RxEventCallback，HAL_UART_RxCpltCallback两个回调中
 */


/* 判断当前接收模式：如果配置为“接收到空闲”模式 */
if (huart->ReceptionType == HAL_UART_RECEPTION_TOIDLE)
{
  /* 恢复为标准接收模式 */
  huart->ReceptionType = HAL_UART_RECEPTION_STANDARD;
```

```
37
38      /* 关闭IDLE中断 */
39      ATOMIC_CLEAR_BIT(huart->Instance->CR1, USART_CR1_IDLEIE);
40
41      /* 若IDLE标志仍置位，则清掉 */
42      if (__HAL_UART_GET_FLAG(huart, UART_FLAG_IDLE))
43      {
44        __HAL_UART_CLEAR_IDLEFLAG(huart);
45      }
46
47    #if (USE_HAL_UART_REGISTER_CALLBACKS == 1)
48      huart->RxEventCallback(huart, huart->RxXferSize);    /* 调用用户注册的空闲事件回
      调 */
49    #else
50      HAL_UARTEx_RxEventCallback(huart, huart->RxXferSize); /* 调用默认弱符号空闲事件
      回调 */
51    #endif
52    }
53    else
54    {
55      /* 标准接收模式：数据已全部收满 */
56    #if (USE_HAL_UART_REGISTER_CALLBACKS == 1)
57      huart->RxCpltCallback(huart);    /* 调用用户注册的接收完成回调 */
58    #else
59      HAL_UART_RxCpltCallback(huart); /* 调用默认弱符号接收完成回调 */
60    #endif
61    }
62
63    //c语言知识
64
65    #if
66
67    #else
68
69    #endif
70
```

再跳转到HAL_UART_RxCpltCallback(huart)：

代码块

```
1   __weak void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
2   {
```

```
  /* Prevent unused argument(s) compilation warning */
  UNUSED(huart);
  /* NOTE: This function should not be modified, when the callback is needed,
           the HAL_UART_RxCpltCallback could be implemented in the user file
   */
}
```

这也是一个弱定义函数，我们在使用它时通常也进行重写

`HAL_UART_RxCpltCallback`：配套 `HAL_UART_Receive_IT()`（固定长度中断接收）。

`HAL_UARTEx_RxEventCallback`：配套 `HAL_UARTEx_ReceiveToIdle_IT()`（接收直到空闲帧或接收满,结合前面IDLE进行理解）。

接下来进行代码编辑：将上文实验中的代码放到重写的回调函数中稍作修改即可

```
/* USER CODE BEGIN PV */
char tx_data[] = "hello";
uint8_t rx_data[100];
uint16_t rx_len;
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart,uint16_t Size) {
  if (huart == &huart1) {
    for (int i = Size;rx_data[i]!=0; i++) {
      rx_data[i] = 0;

    }
    HAL_UART_Transmit_IT(&huart1, (uint8_t *)tx_data,Size:strlen(tx_data) );
    HAL_UARTEx_ReceiveToIdle_IT(&huart1, (uint8_t *)rx_data, Size:100);
  }
}
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
  if (huart == &huart1) {
    HAL_UART_Transmit_IT(&huart1, (uint8_t *)rx_data, Size:strlen(rx_data) );
    HAL_UART_Receive_IT(&huart1, (uint8_t *)rx_data, Size:5);
  }
}
//这两个只需要处理一个,看需要啥了,而且这都是每次结束后都立刻开始下一轮接收的写法
/* USER CODE END PFP */
```

可以发现在每个回调函数中都调用了对应的接收中断使能函数，这是因为每次触发该中断时会将其失能，所以我们在每次回调函数的最后重新使能中断

```
  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART1_UART_Init();
  /* USER CODE BEGIN 2 */
  HAL_UARTEx_ReceiveToIdle_IT(&huart1, (uint8_t *)rx_data, Size:100);
  //HAL_UART_Receive_IT(&huart1, (uint8_t *)rx_data, 5);
  //上面两个写法根据需求来,同一个UART只能用一个,前面我们写回调时自动每次结束后重启,所以不需要在主循环里再启动了
  /* USER CODE END 2 */

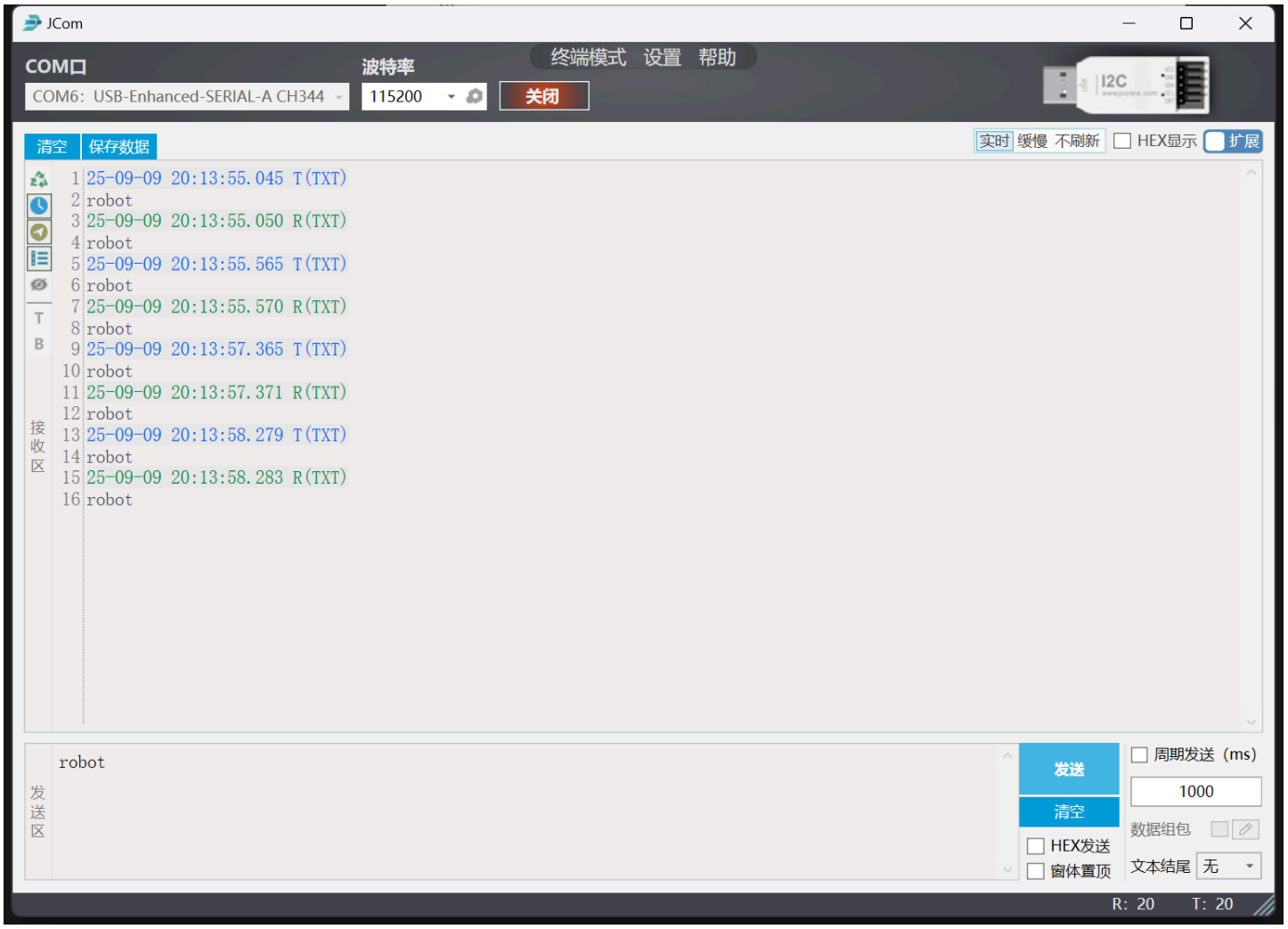
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    HAL_Delay(500);
  }
  /* USER CODE END 3 */
}
```

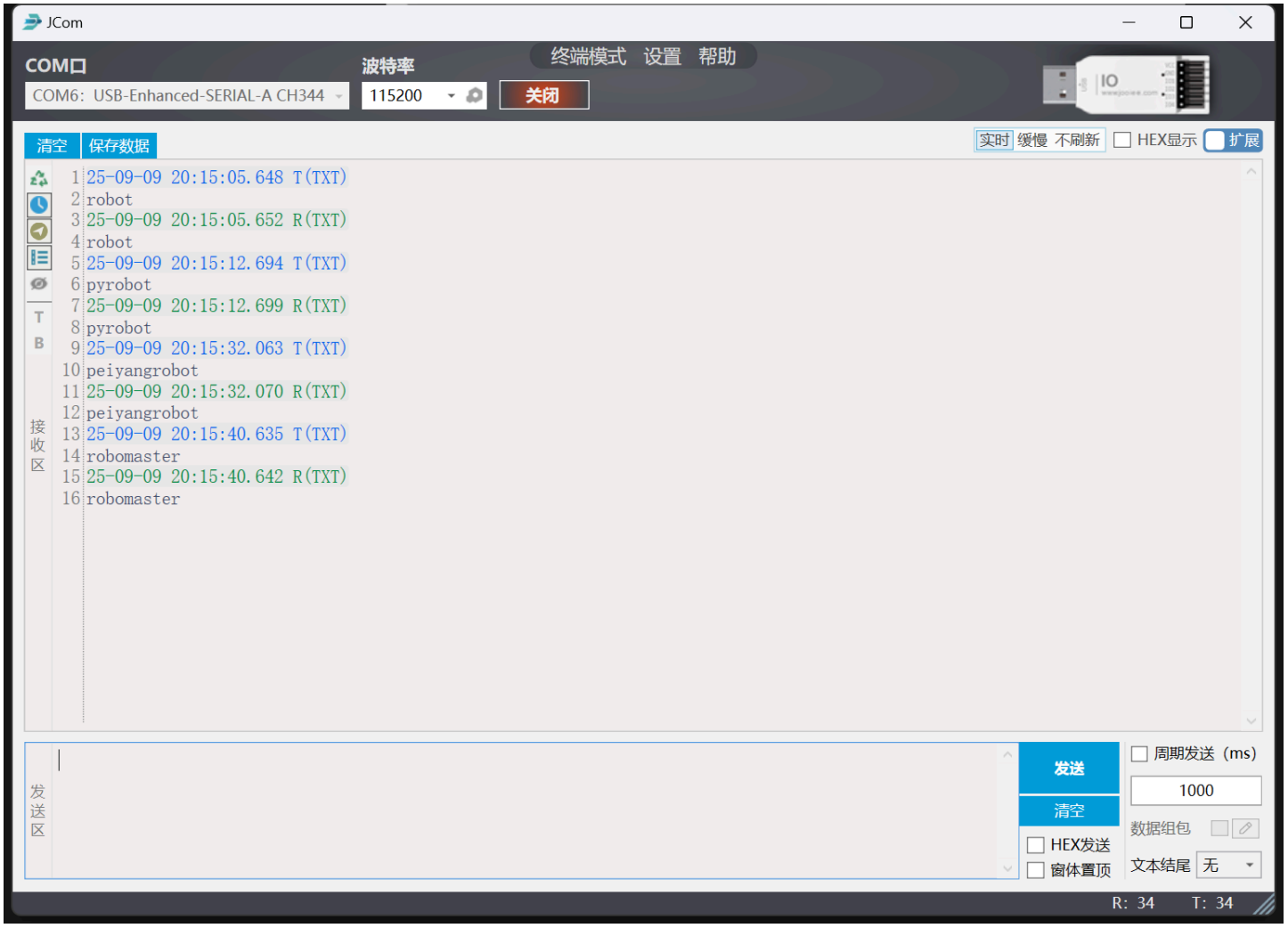
同时在main函数中我们也要使能一次中断，由于只需要使能一次，所以并不需要放到while循环中（图中仅做示范，HAL_UARTEx_ReceiveToIdle_IT与HAL_UART_Receive_IT同个串口只能使能一个）

这里看一下实验现象：

HAL_UART_Receive_IT：



HAL_UARTEx_ReceiveToIdle_IT：



关于接收,推荐将接收和处理分离,也就是在接收回调里只是把数据搬运好,然后在后面处理,比如要解析数据帧,在接收回调里只是把数据搬运走,然后校验解析什么的交给别的任务完成

也推荐另一种写法,一点点收数据,然后压进一个大缓冲区队列里(也别太大),然后从队列里往外弹数据来解析,此时我们的解析就变成了:弹队列直到找到帧头(或者帧尾然后弹出),然后尝试解析校验一个数据帧长度的内容,如果符合,认定为一个合法数据帧,反之,弹掉刚才那个开头继续找

另外,关于帧头帧尾,有时候会被大小端整一下,用的时候注意一下吧

大端模式：数据的高字节存储在内存的低地址，低字节存储在高地址。例如，32位整数0x12345678在大端存储中顺序为12 34 56 78

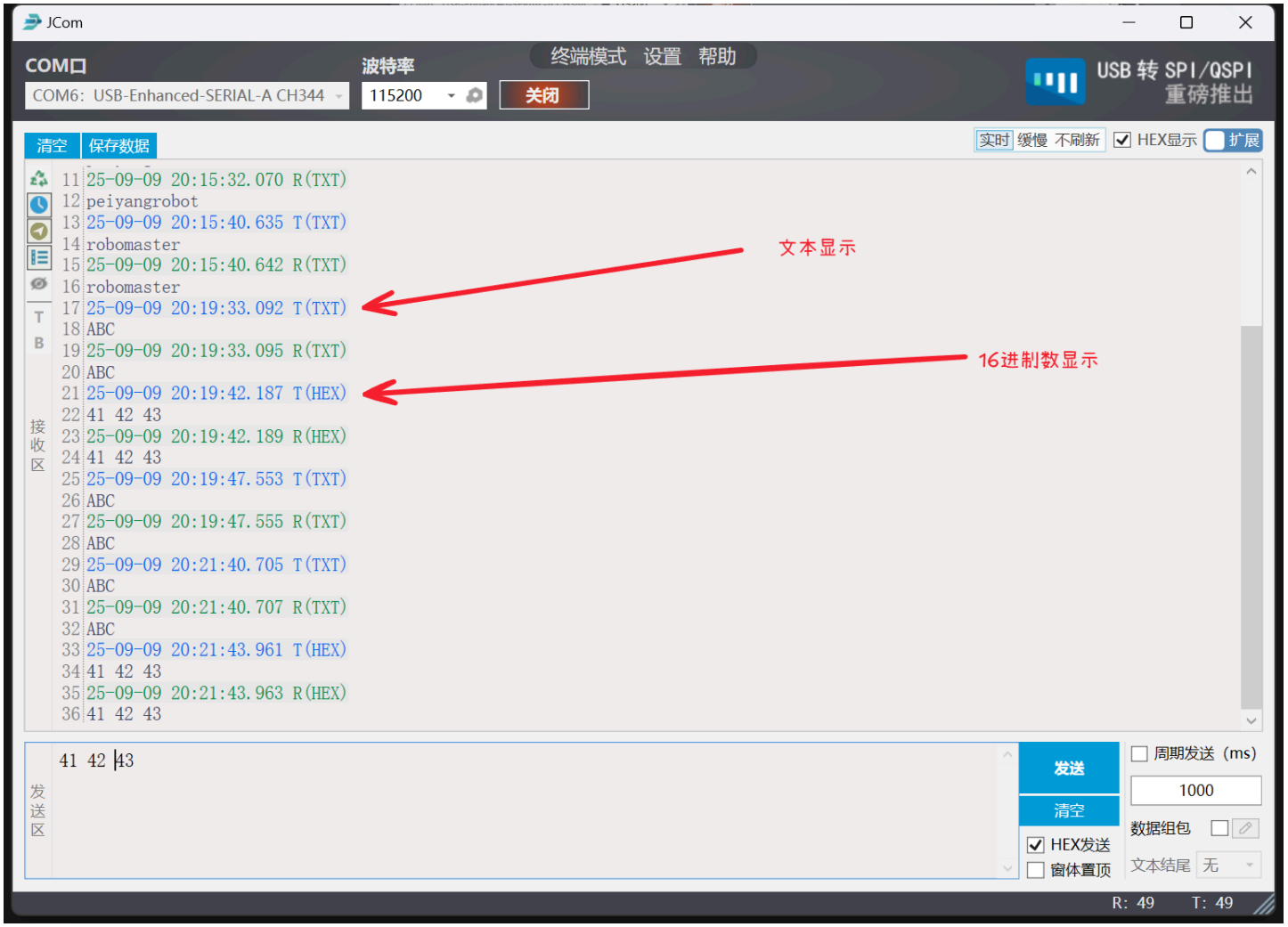
小端模式：数据的低字节存储在内存的低地址，高字节存储在高地址，同样的整数在小端存储中顺序为78 56 34 12

最后再学一个16进制(HEX)发送：

无论是char 还是uint8_t 均占8位，一个16进制数不难算出占用4位，故我们发送16进制数据时要进行成组发送，一个两位16进制数为一组

通过查ASCII表我们可以得知字符ABC对应的16进制数分别为41，42，43

如图进行发送得到的现象：



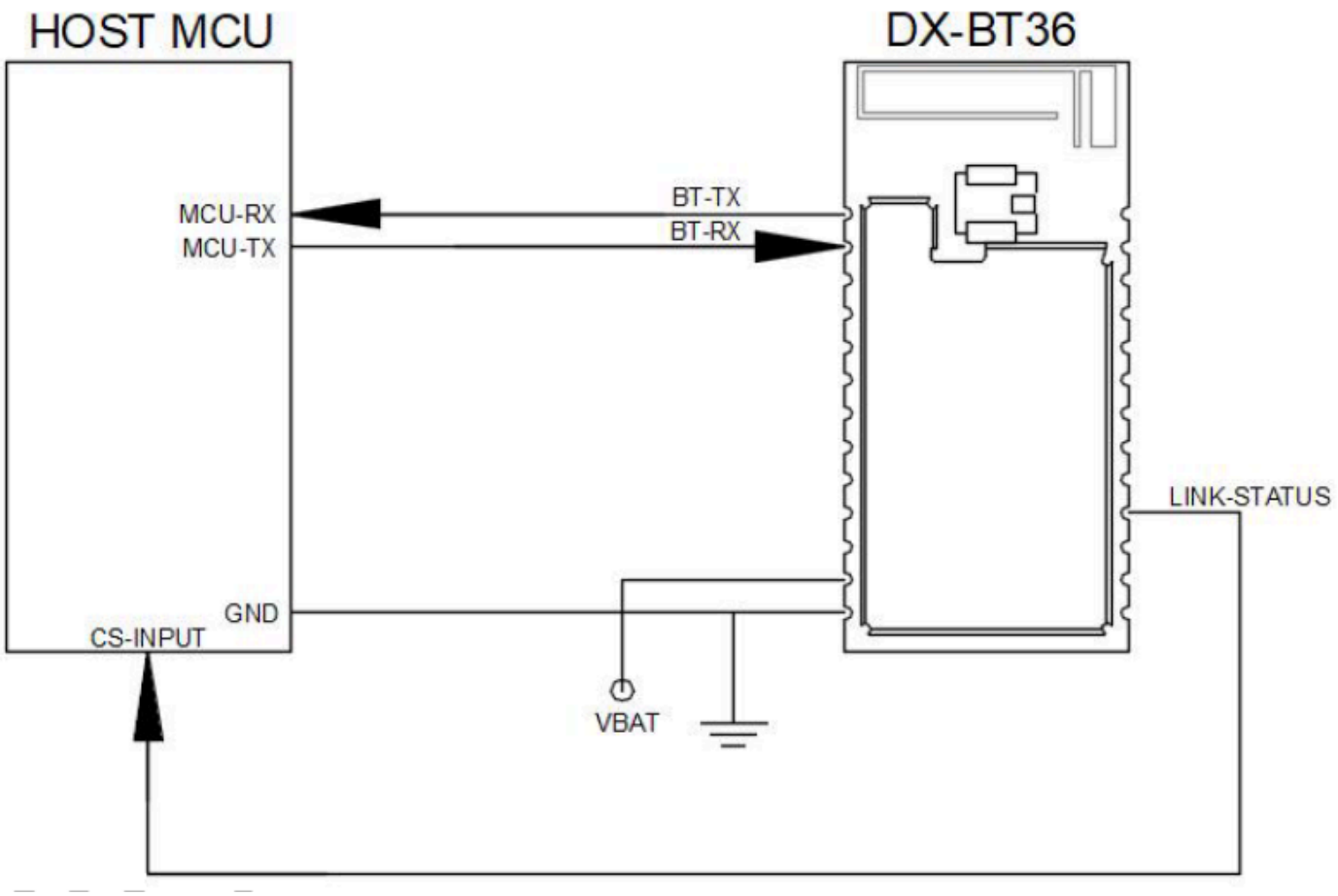
蓝牙使用

基于上文，我们实现了UART的有线通信，但我们常常有无线通信的需求，例如无线鼠标，无线耳机，遥控小车等，这时我们可以借助蓝牙模块的帮助

这里我们选用BT36蓝牙模块作示例，附商家提供资料：

DX-BT36资料包.zip
4.88MB

接线图：



简而言之，就是把原本接到电脑的线接到了蓝牙模块的对应位置，之后电脑或手机通过自带的蓝牙进行连接

蓝牙模块需要供电，所以这里会连一个VCC

AT指令（此部分在上文附件资料包-02-pdf中，此处附）

DX-SMART

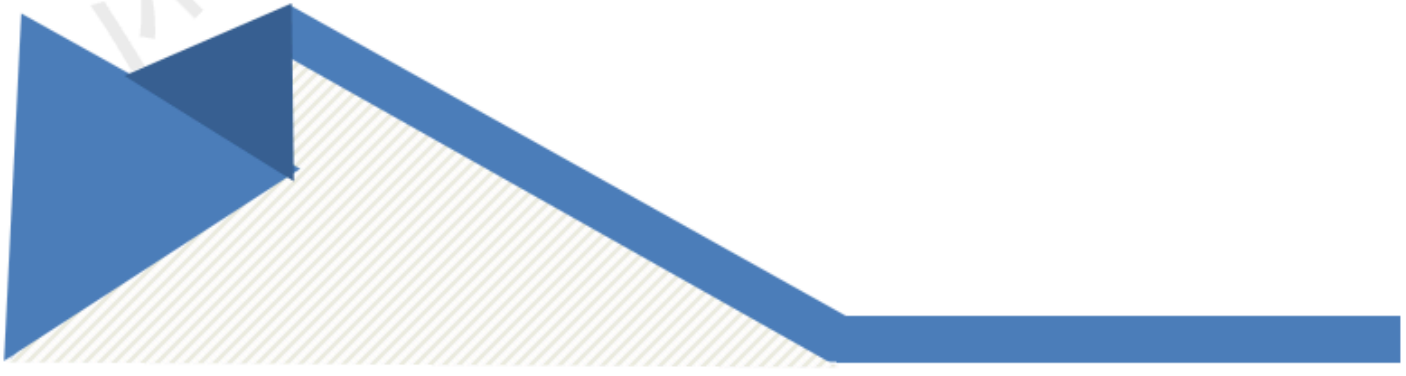
DX-BT36

串口应用指导

版本：2.2

日期：2024-06-05

深圳大夏龙雀科技有限公司



使用蓝牙模块前，我们需要使用AT指令对其进行参数配置

以下为AT命令格式：

4.1. 命令格式说明

AT+Command<param1，param2，param3> <CR><CF>

- 所有的指令以 AT 开头，<CR><LF>结束，在本文档中表现命令和响应的表格中，省略了<CR><LF>，仅显示命令和响应。
- 所有 AT 命令字符都为大写。
- <>内为可选内容，如果命令中有多个参数，以逗号“，”隔开，实际命令中不包含尖括号。
- <CR>为回车字符\r，十六进制为 0X0D。
- <LF>为换行字符\n，十六进制为 0X0A。
- 指令执行成功，返回相应命令以 OK 结束，失败返回 EEROR=<>，“<>”内容为对应错误码（请参考 5.7）。

科技有限公司

命令一览表：（根据文档ai提取，如有错误以压缩包中的文档为准）

指令	功能	说明
AT	测试指令	用于测试串口
AT+VERSION	查询版本号	依据不同的模块与定制需求版本会有区别
AT+LADDR	查询 MAC 地址	-
AT+NAME	设置/查询蓝牙名	默认名称：BT36
AT+NAMAC	设置/查询蓝牙名称+MAC	默认：0（关闭）
AT+STOP	设置/查询串口停止位	默认：0（1停止位）
AT+PARI	设置/查询串口校验位	默认：0（无校验）

AT+BAUD	设置/查询波特率	默认：3（9600）
AT+DISC	断开连接	只能在透传模式下使用，且只能由串口端发送有效，手机端发送无效
AT+RESET	软件重启	-
AT+DEFAULT	恢复出厂设置	-
AT+UUID	设置/查询服务 UUID	默认服务 UUID: 0xffe0
AT+CHAR	设置/查询模块通知/写入 UUID	默认值：0xffe1
AT+WRITE	设置/查询模块写入 UUID	默认值：0xffe2
AT+MUUUD	设置/查询主机服务 UUID	默认主机服务 UUID: ffe0
AT+MWRITE	设置/查询主机写入 UUID	默认值：ffe2
AT+NOTI	设置/查询通知上位机连接状态	默认：0（不通知）
AT+INQ	手动搜索蓝牙设备	详情请看 5.4.2
AT+CONN	手动连接蓝牙设备	详情请看 5.4.3
AT+CONA	连接远端指定蓝牙	详情请看 5.4.4
AT+BIND	设置绑定蓝牙地址	详情请看 5.4.5
AT+AUTOCONN	设置/查询模块连接模式	默认关闭
AT+CLEAR	清除记忆蓝牙地址	该指令只能在 AT+AUTOCONN 为 1 的时候生效清空 AT+BIND、AT+AUTOCONN 这两条指令连接之后的蓝牙地址
AT+FILTER	设置/查询蓝牙设备名称	设置过滤蓝牙名称后，模块只连接该名称的蓝牙设备
AT+TIME	设置/查询 INQ 搜索** 搜索时间	默认：10（1000ms）
AT+PWRM	设置/查询工作模式	默认：1（正常工作模式）
AT+ADVI	设置/查询广播时间间隔	默认：5（546ms）

at命令详解内容太多，难以将所有内容都粘贴下来，请参考文档第五章，这里我进行一次使用示例，并把相应资料放上来：



EEROR=<>中错误码码的详细信息列举如下：

返回值	错误信息说明
101	参数长度错误
102	参数格式错误
103	参数数据异常
104	指令错误

5.1.1. 测试指令

功能	指令	响应	说明
测试指令	AT	OK	用于测试串口

5.1.6. 设置\查询—串口停止位

功能	指令	响应	说明
查询串口停止位	AT+STOP	+STOP=<param>	< param>序号 0：1 停止位 1：2 停止位 默认值：0
设置串口停止位	AT+STOP<param>	+STOP=<param> OK	

5.1.7. 设置\查询—串口校验位

功能	指令	响应	说明
查询串口校验位	AT+PARI	+PARI=<param>	< param>序号 0：无校验 1：奇校验 2：偶校验
设置串口校验位	AT+PARI<param>	+PARI=<param> OK	

5.1.8. 设置\查询—串口波特率

功能	指令	响应	说明
查询波特率	AT+BAUD	+BAUD=<baud>	<baud>波特率对应序号 1: 2400 5: 38400 2: 4800 6: 57600 3: 9600 7:115200 4: 19200 默认值: 3(9600)
设置波特率	AT+BAUD<baud>	+BAUD=<baud> OK	

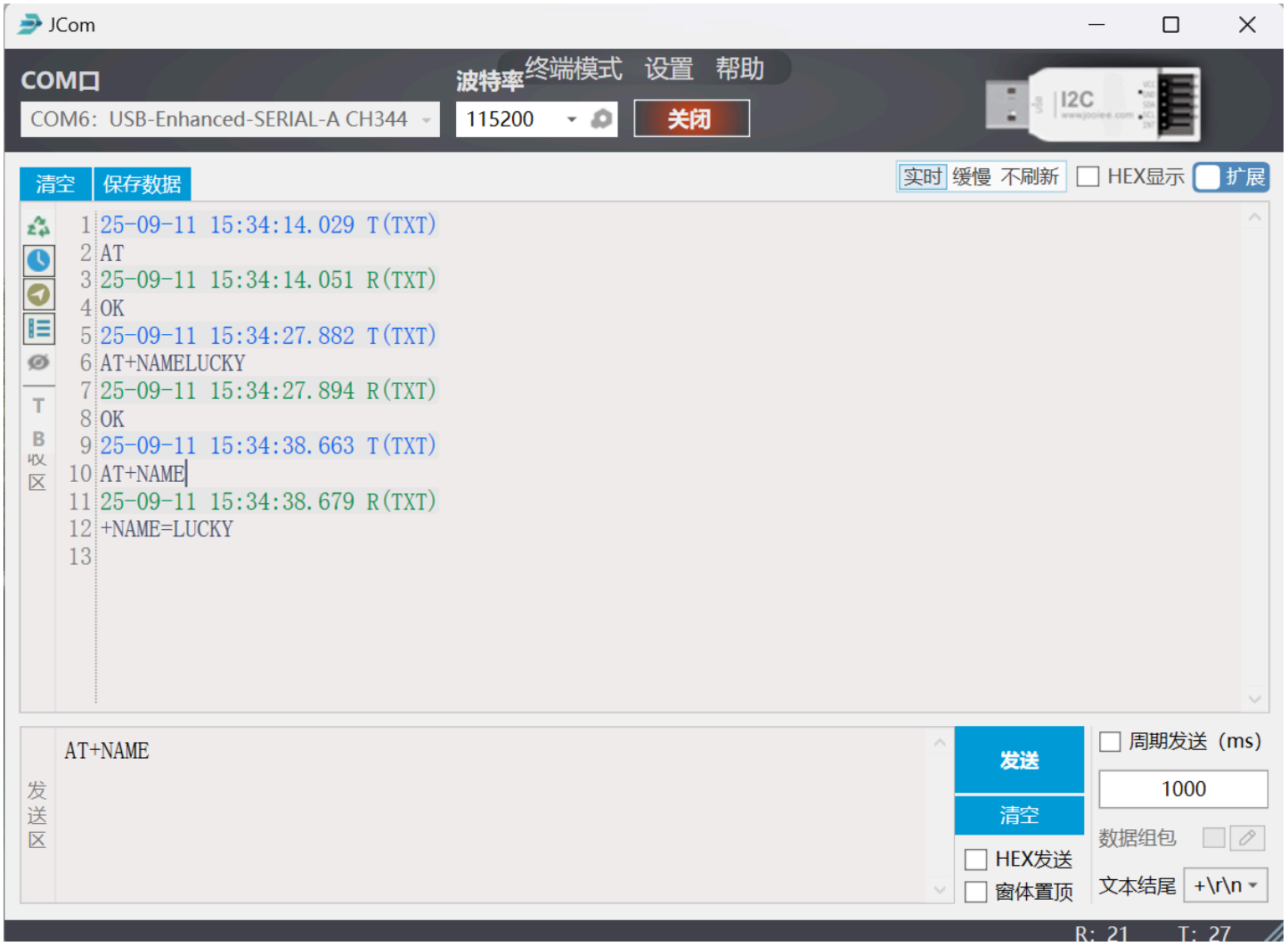
备注：

设置完该指令后需重启生效。

需要注意在更改时不要加空格，如上18行报错

设置波特率不是立即生效，重启后（断电再供电）生效

为了方便我们用手机进行连接，给其自定义蓝牙名，这个也是重启后生效



蓝牙通信

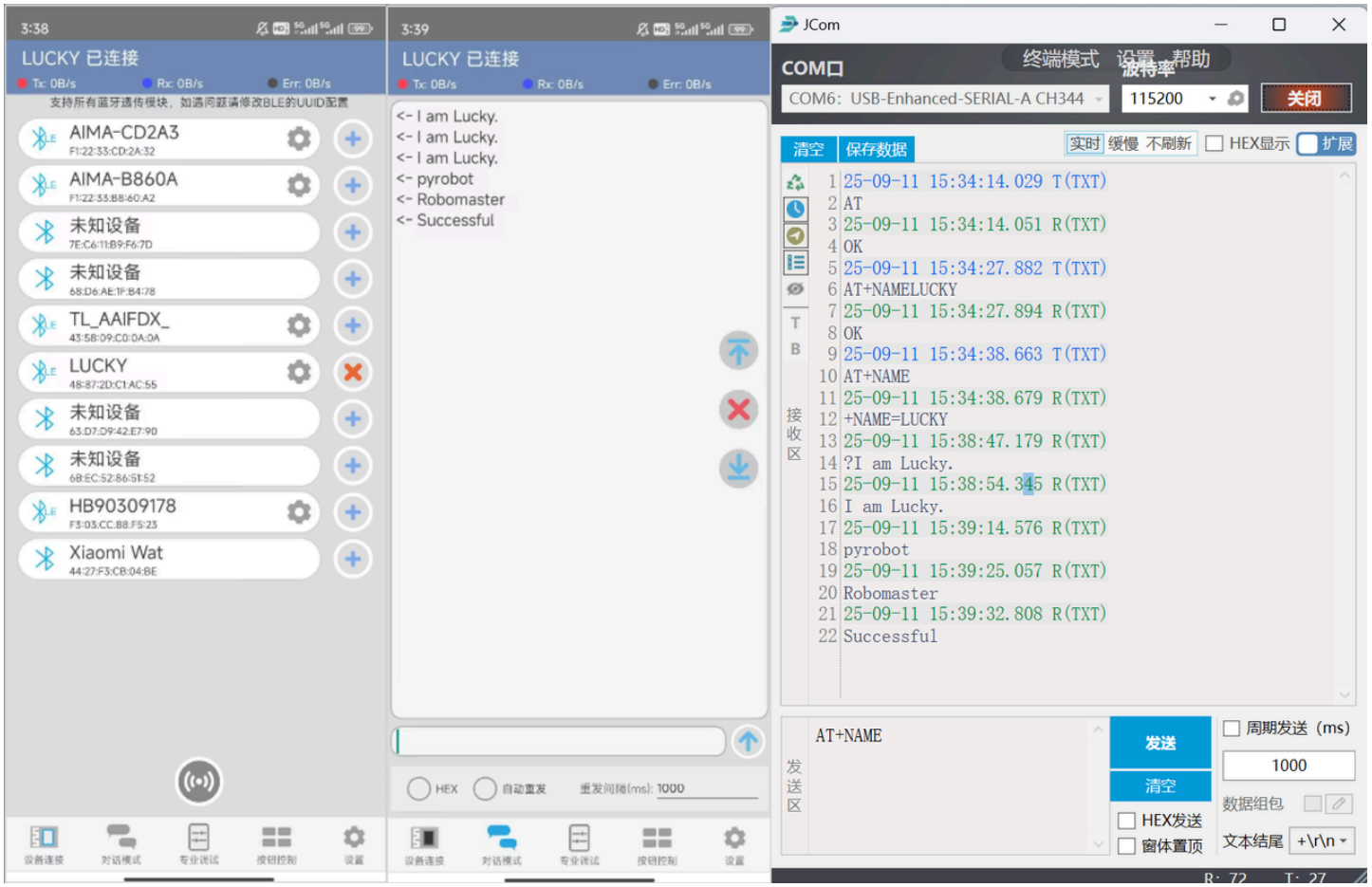
下文为了调试过程清晰，蓝牙模块直接连接电脑，通过JCOM串口助手查看数据，连接单片机时只需将对应线连到单片机的对应IO口即可。

对话模式

重新插拔usb转ttl，

Android：在手机app蓝牙调试器（附apk）上找到LUCKY并连接

xnj.lazydog.btcontroller.zip
2.28MB



如图成功连接后进入对话模式即可发送消息

备用链接：https://wwed.lanzout.com/ifwl835xgk3g

密码:e1ty

iOS：iOS上没有像安卓那样功能强大的蓝牙调试器，只能用厂家提供的“丐版”



由于我手上没有不能吃的apple，所以下文使用app仅是厂商提供的同款，经比较，基础功能相同，Android上有更多扩展功能，先不用



如图成功连接后进入对话模式即可发送消息

至此，我们就成功通过蓝牙让手机和电脑间互发了消息，如果希望手机与单片机之间通信只需要将蓝牙的对应接口接到单片机上即可

按键与专业模式：

iOS的app并不支持此功能，如果想要尝试类似的功能：

~~只需要再买个手机~~

事实上可以通过两个蓝牙模块，一主一从进行，but，绝大多数的电脑不能触屏，而且JCOM对于此方面尤其是触控的适配并不好，也只是可以体验的地步，想作为一个遥控去控制实时性要求高，动作较为复杂的机器（如校内赛的小车）还是难以有较好的体验

但也不必要真的去买个手机，队内赛时会协调，避免有全组队员都没有Android的情况

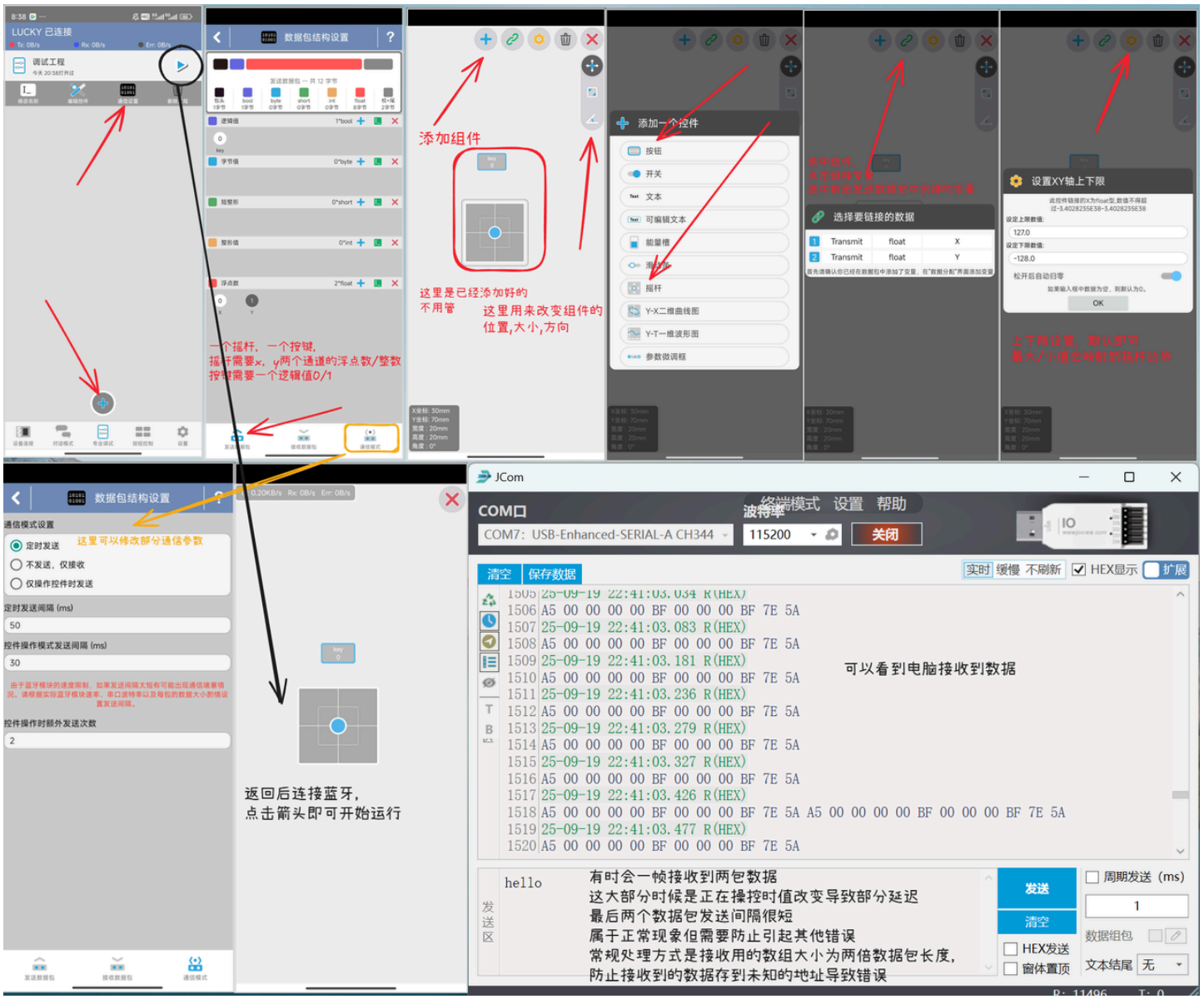
这部分仅针对Android用户编写，iOS用户可以直接进入下一个模块，关于如何进行一主一从的蓝牙通信，可以参考上文手册进行使用

按键控制：



专业调试：

现在先假设我们有一个遥控小车，它要实现的基本功能有前后左右的移动与转向/原地转圈，很直观的，我们希望其有一个摇杆（控制前后左右），以及一个按键（控制是否原地转圈），控制前后左右时，我们认为摇杆的值为期望的速度，控制转圈时，我们则只需要告诉它是/否，即1/0，明白了这些，我们开始进行专业模式的使用



补充：

1.上图第二行第一个图中的控件操作时额外发送次数是为了避免蓝牙传输过程中出现错误/丢失数据等情况，进行的额外发送

2.专业模式时，app规定了包头，包尾与校验，其规定如下：

包头，包尾与校验是为了接收方判断自己收到的数据是否正常，确保数据的正确传输



于接收数据包而言，其配置方式与发送数据包类似，运行时结果会经过解析后可视化在所添加的组件中，在经过下一模块VOFA/JCOM的学习后，可以尝试独立探索其用法.

(多捣鼓捣鼓专业模式,队内赛要用!)

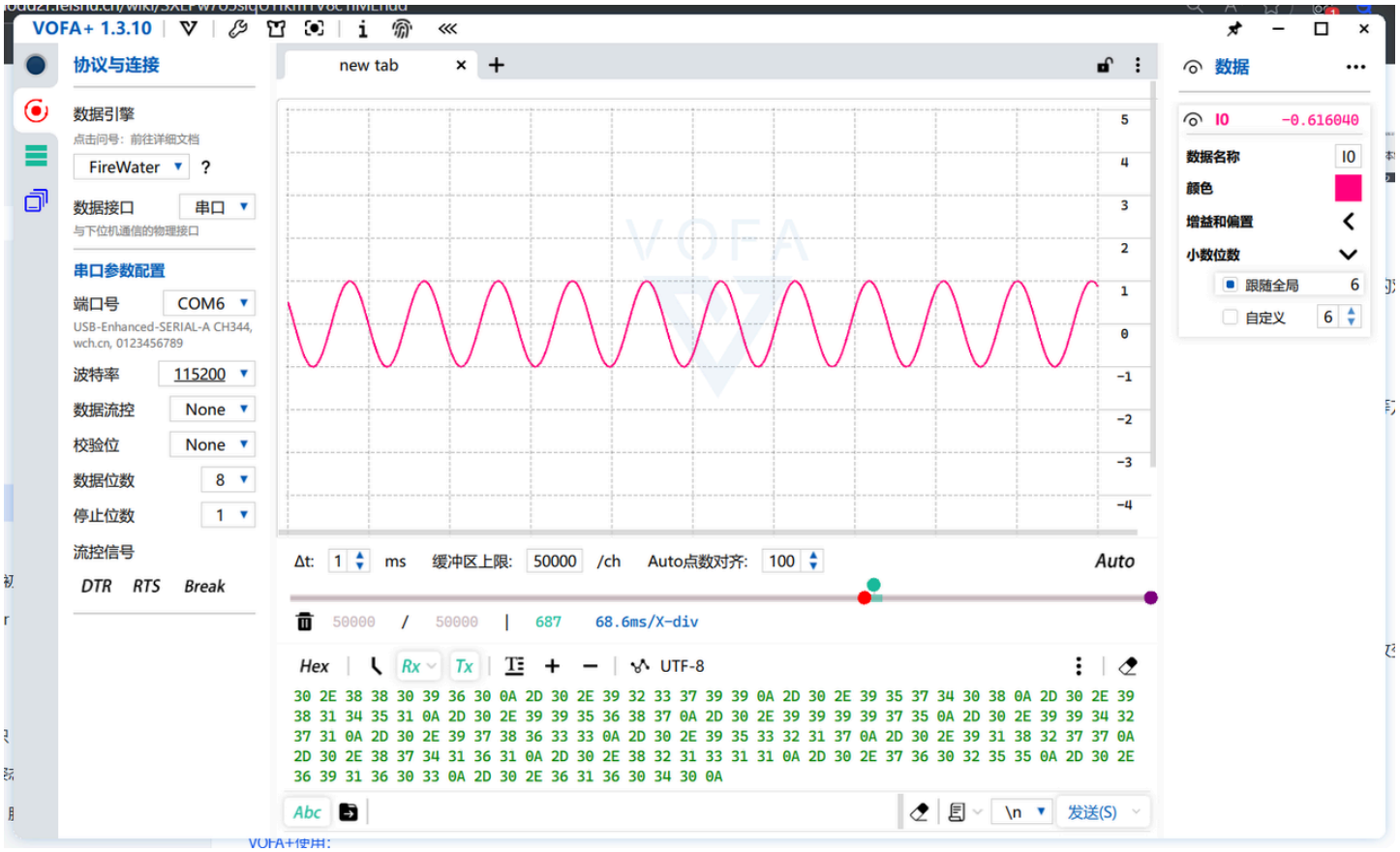
VOFA+(非必要章节):

本章节不是必要内容,JCOM也可以解决这些问题,所以可以跳过,不过有时候JCOM会出现一些抽风现象比如识别不了串口,瞎胡乱切帧啥的,所以,如下文括号所言

（本来打算只用vofa的，但我被vofa的十转软件蛊坑了，于是选择把两个都写一下，实际使用中，如果遇到此类问题，更多的调试工具可以让我们不至于手足无措）

介绍:

VOFA是一个兼具美观的多功能串口助手，其显著的优点是能将数据以波形，模型等方式进行可视化，优化我们的调试体验。如图：这里我生成了正弦函数的波形



下载：

安装包：

vofa+_1.3.10_x64_installer.exe
27.60MB

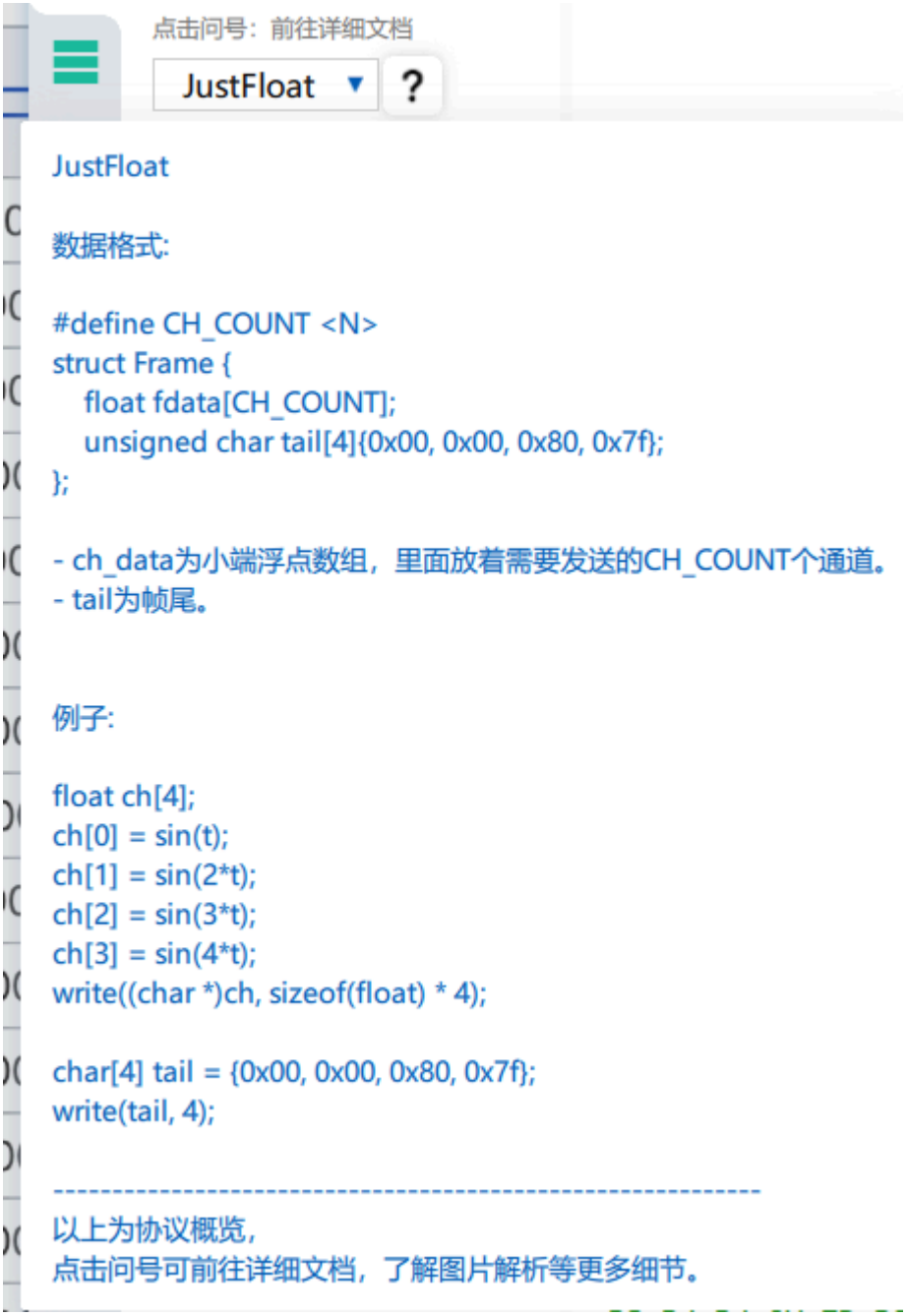
官网：

VOFA+下载中心

使用：

JustFloat协议：

协议格式如下，正如其名，这个协议只能用来解析float类型的数据，上文将printf进行重定向是因为需要使用到printf的字符处理功能，此时没有字符，所以我们不再需要重定向



如上，该协议的要求是先发一个float数组，再发一个四字节的帧尾，可以选择将其组成一个结构体一起发送，也可以按照顺序依次发送，这里我们选择组成一个结构体

代码块

```
#define CH_COUNT 2

typedef __packed struct{
    float fdata[CH_COUNT];
    uint8_t tail[4];
}vofa_frame_t;

vofa_frame_t vofa_frame;
```

通过以上代码完成结构体变量的声明和定义，这里解释一下__packed的作用：

C语言的四字节对齐规则：

结构体的首地址为0，其所有 4 字节宽的成员必须落在能被 4 整除的地址上，结构体总大小再补到 4 的整数倍；不满足就插0，插到满足为止。

代码块

```
typedef struct {
    uint8_t a;// 占用0，补齐1~3
    int b;//占用4~7
    uint8_t c;//占用8
};//结构体总大小为9，不是4的整数倍，补三字节到12

//当换一种写法时
typedef struct {
    int b;//占用0~3
    uint8_t c;//占用4
    uint8_t a;// 占用5
};//结构体总大小为6，不是4的整数倍，补两字节到8
```

如上所示，在四字节对齐的规则下，编译器会隐式的补充字节，对于双方通讯很不友好，并且会占用额外的空间，于是我们常常使用__packed进行单字节对齐，即取消编译器对结构体/联合体的任何填充（padding），让成员之间“紧挨着”排列

回到代码，根据协议，我们需要给帧尾进行赋值，同时不断发送要显示的值。

代码块

```
    vofa_frame.tail[0] = 0x00;
    vofa_frame.tail[1] = 0x00;
    vofa_frame.tail[2] = 0x80;
    vofa_frame.tail[3] = 0x7f;
    //按照帧头帧尾要求赋值
    double t = 0.0f;//这里f是float的意思,其实直接写0也行,这里又不会引起歧义,但有时候会自动转成double,所以遇到时注意一下吧

    while(1)
    {
        x+=0.1;

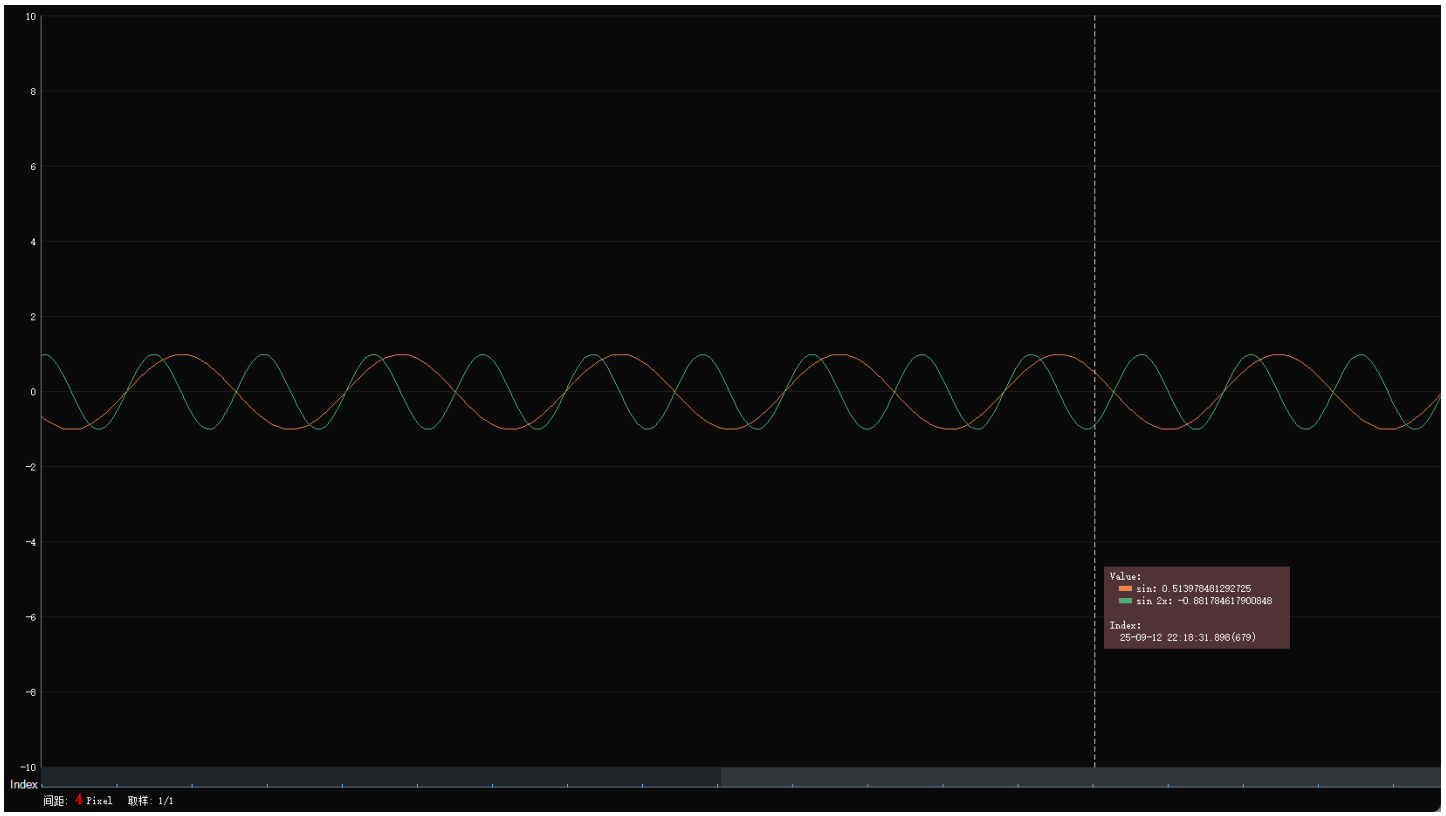
        vofa_frame.fdata[0] = sin(X);
        vofa_frame.fdata[1] = sin(2*x);

        HAL_UART_Transmit_IT(&huart1,(uint8_t*)&vofa_frame.sizeof(vofa_frame));
        //取缓冲区的地址然后转成对应类型的指针传进去

        HAL_Delay(50);
```

```
19      }
20
```

这时我们就已经成功的完成了代码编写，我这里输出了sinx与sin2x的波形，但由于十转软件蛊，这里没有vofa下实验成功的截图，放一张JCOM上输出的乐一乐



JCOM进阶使用

介绍：

JCOM是一个十分全面的串口助手，我们在前面用它的基本功能进行了多次实验，今天对其扩展功能进行部分讲解，在这里和vofa相同，只讲解显示波形功能，更多的有兴趣可参阅官方文档继续学习：

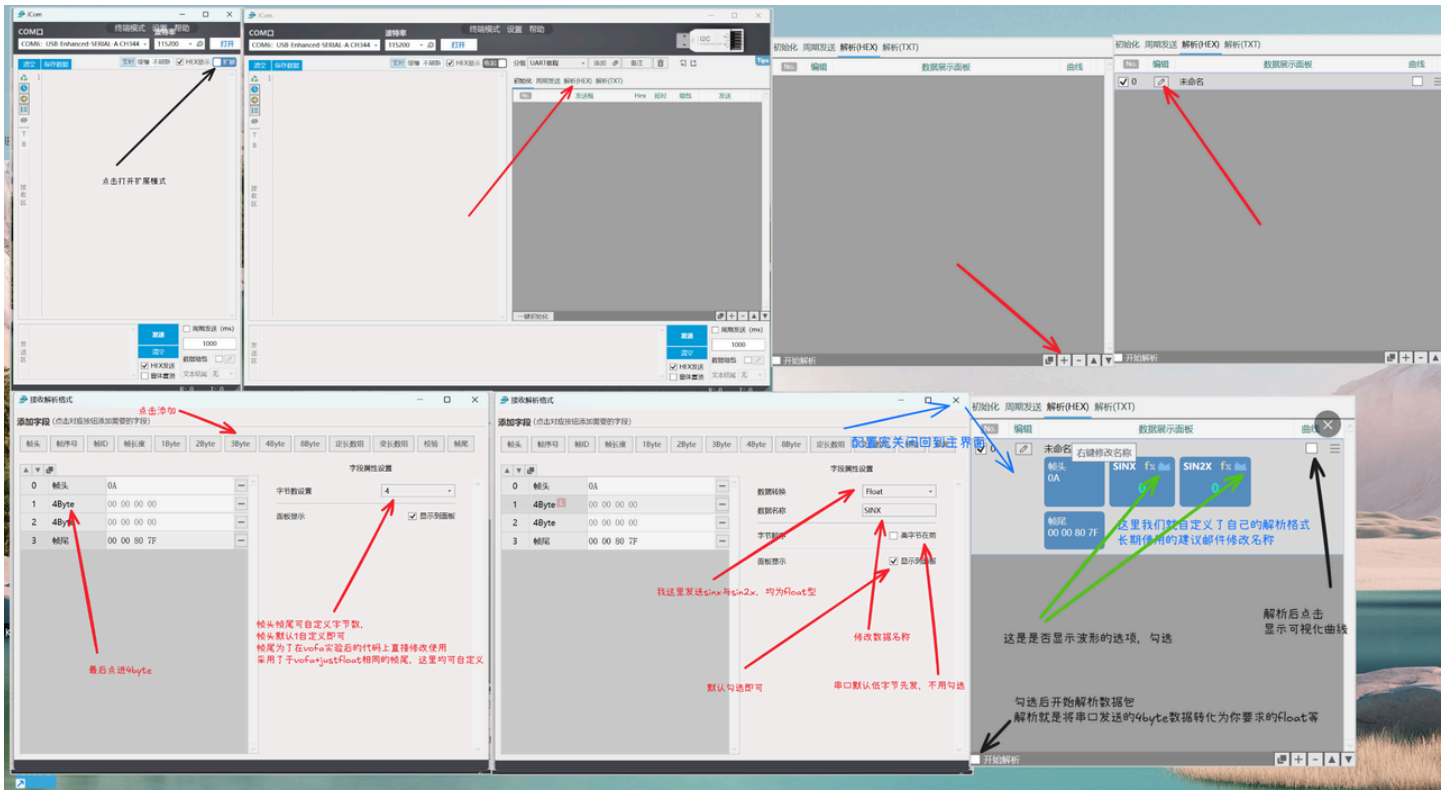
JCOM

相比于vofa，它可以自定义协议，但必须含有帧头“即数据起始的固定标志”，无须含有帧尾

使用：

JCOM从零创建一个自定义解析如下：（这里我的帧尾和vofa+justfloat协议一致，但没有看vofa的也不用担心，这部分完全自定义，只需要你的代码与软件配置一致即可。

（这里图中的最后一幅图应该在编写代码后放出来，不小心放多了，可以先无视）



构建结构体，这里要按照自己设定的数据帧格式进行构造

代码块

```
#define CH_COUNT 2

typedef __packed struct{
    uint8_t head;
    float fdata[CH_COUNT];
    uint8_t tail[4];
}vofa_frame_t;

vofa_frame_t vofa_frame;
```

通过以上代码完成结构体变量的声明和定义,__packed用于取消四字节对齐

回到代码，根据协议，我们需要给帧尾进行赋值，同时不断发送要显示的值。

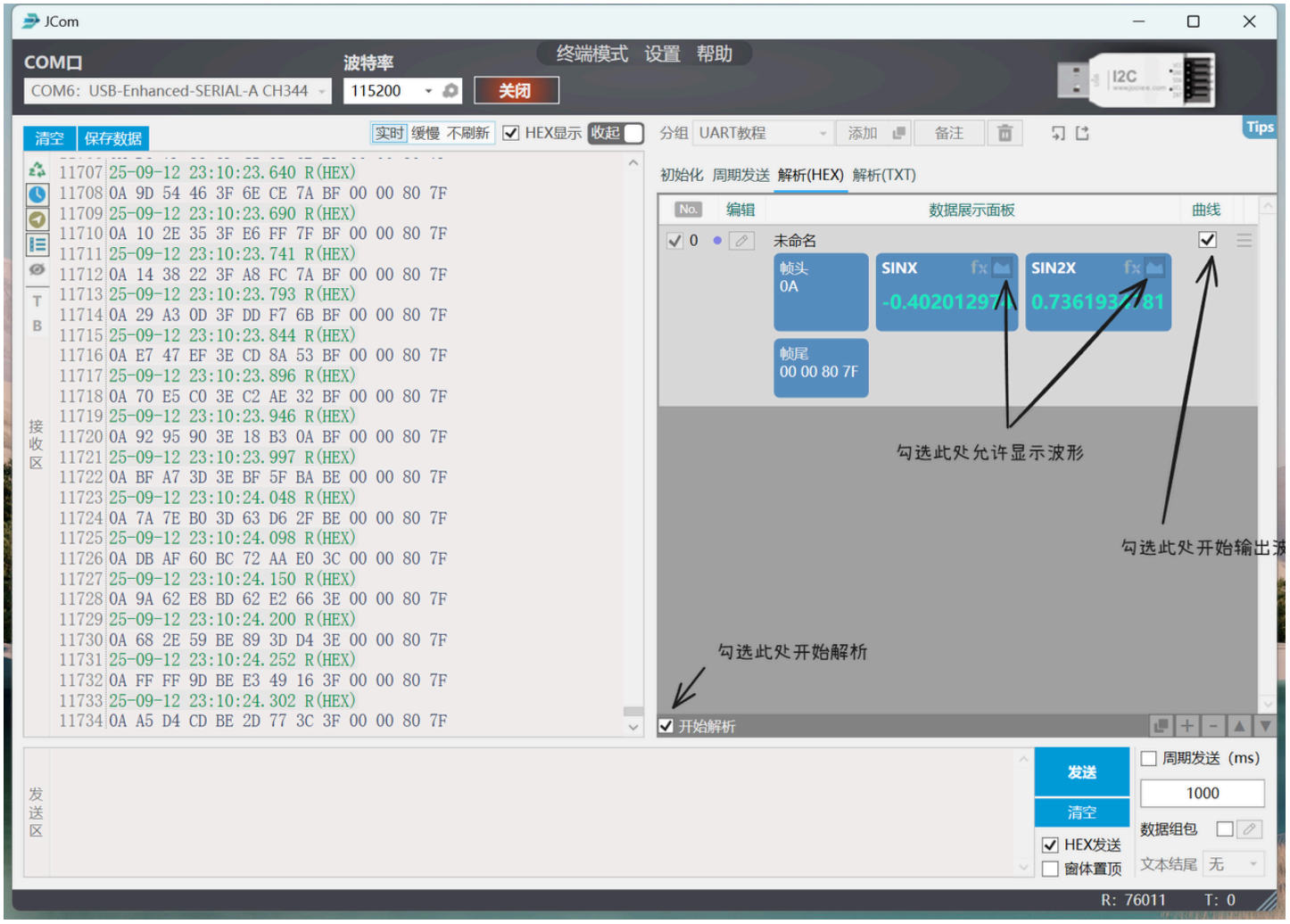
我们这里实验输出sin函数，所以需要#include "math.h"

代码块

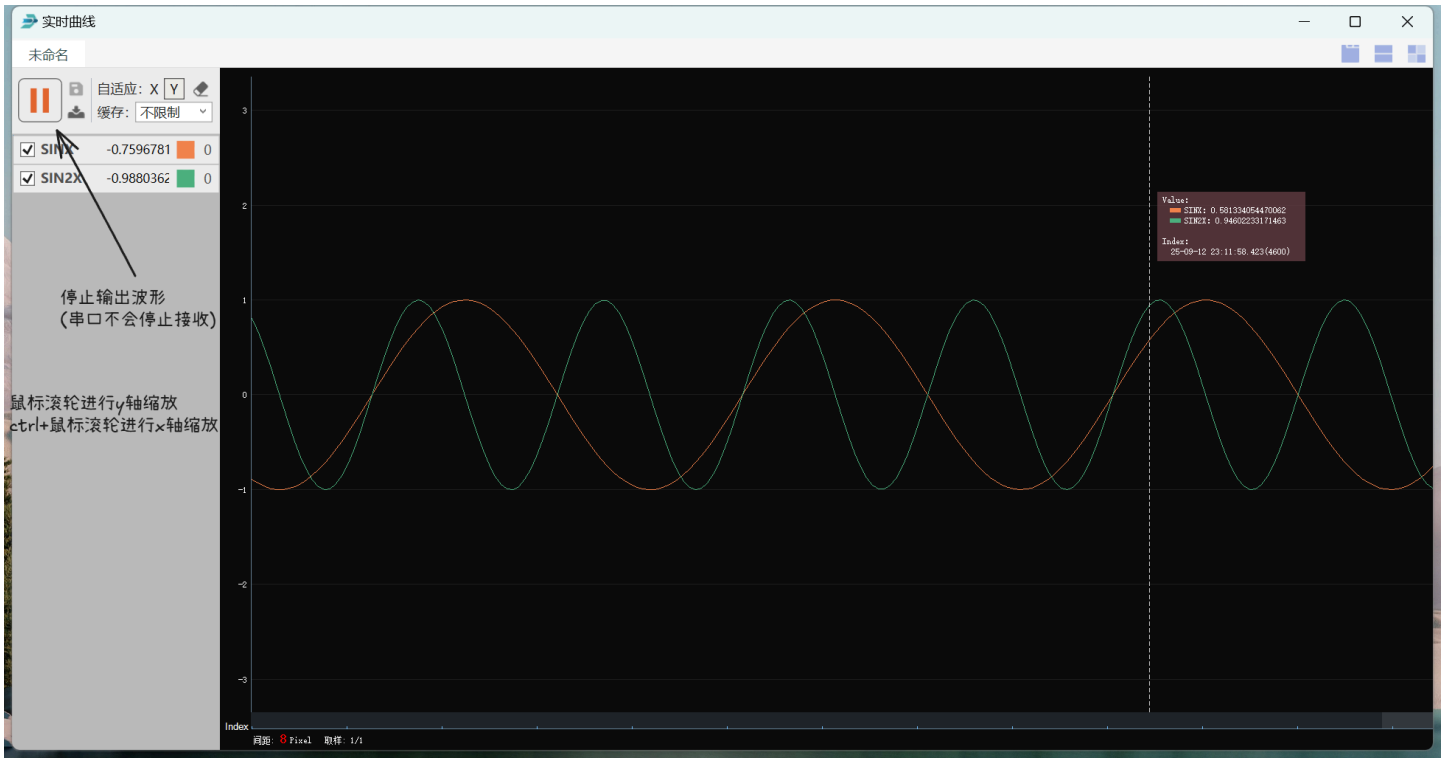
```
    vofa_frame.head = 0x0a;
    vofa_frame.tail[0] = 0x00;
    vofa_frame.tail[1] = 0x00;
    vofa_frame.tail[2] = 0x80;
    vofa_frame.tail[3] = 0x7f;
    //按照帧头帧尾要求赋值,这次是按照自己约定的方式
```

```
7       double t = 0.0f;
8       while(1)
9       {
10          x+=0.1;
11
12          vofa_frame.fdata[0] = sin(X);
13          vofa_frame.fdata[1] = sin(2*x);
14
15          HAL_UART_Transmit_IT(&huart1,(uint8_t*)&vofa_frame.sizeof(vofa_frame));
16
17          HAL_Delay(50);
18      }
19
```

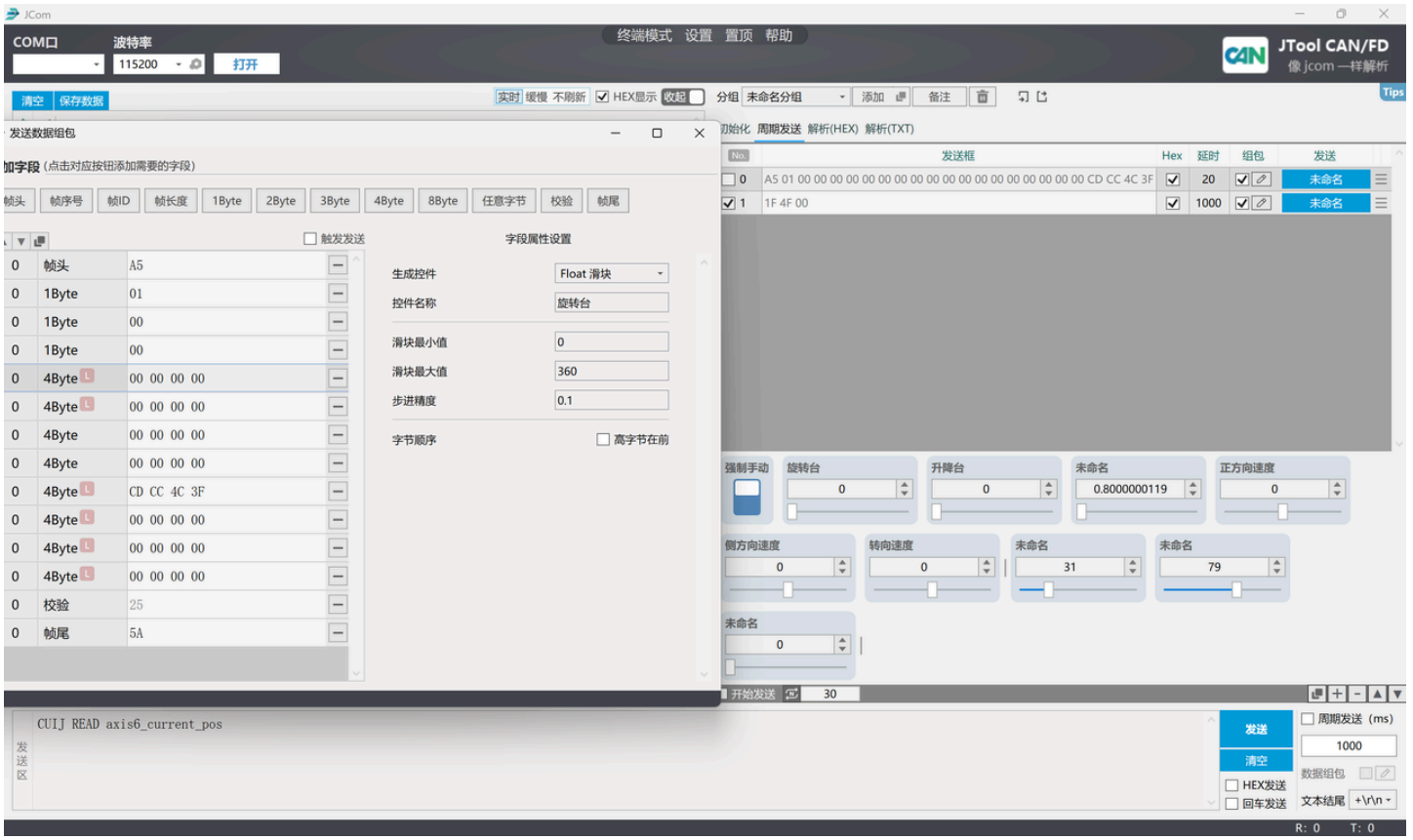
烧录代码，打开JCOM，



当做完这些，我们就可以看到sinx与sin2x的波形了



以此类推,我们还可以在JCOM里打包并发送给下位机,依旧是刚才的位置,然后右下角加号



在这里可以按照需求打包,可以添加一些简单的控件,但是不能像那个手机蓝牙工具那样灵活,下面那个循环开了是循环次数,不开是无限循环

TASK:

实战

1、轮询接收和发送定长数据：

使用HAL_UART_Transmit，HAL_UART_Receive，用电脑发送消息控制两个小灯亮灭变化，当小灯亮灭改变时发送哪个小灯亮/灭了

tip:可以给小灯用十六进制数编号，用0，1表示亮灭

2、中断收发不定长数据：

用电脑发送消息控制两个小灯亮灭变化，当小灯亮灭改变时发送哪个小灯亮/灭了，要求一条消息可同时控制两个小灯,接收形式自定,可以直接接收然后拼数据或者利用idle收不定长都可以,只要是中断收发就行

3、中断收发定长数据：

让两个小灯周期性亮灭,用手机蓝牙工具的专业调试功能来控制两个小灯在1s内亮的时间，控件形式自定,要求小灯以1s为周期变化,可以分别控制两个小灯亮起的时间(ms)

4、使用VOFA+或JCOM或Android手机端蓝牙调试器app显示sin x,sin 2x,cos x,cos 2x的图像

思考

1、串口轮询接收与发送，以及接收不定长数据时查看的是哪个寄存器的哪位，当总线空闲时，总线电平为

2、串口收发数据时数据在硬件中的传输路径

3、在cubemx生成代码的默认情况下，调用HAL_UART_Transmit_IT,HAL_UART_Receive_IT,HAL_UARTEx_ReceiveToIlde_IT后分别会在什么情况下进入中断

tip：可以尝试使用打断点，单步调试等方式进行查看

4、波特率反映通信的什么，双方波特率必须一致吗，如果不一致会发生什么，因为什么